Design and Implementation of an Agentic Demand Forecasting and Order Confirmation System for Sri Lankan B2B Retail

Madusanka R.R.V

(IT22332530)

BSc (Hons) Degree in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
Madusanka R.R. V	IT22332530	

The supervisor/s should certify the proposal report with the following declaration.

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

26.04.2026

…….. ……

Signature of the supervisor: Date

…….. ……

Signature of the co-supervisor: Date

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported me throughout the course of this research and the preparation of this thesis. First and foremost, I am deeply grateful to my supervisor, Ms. Samanthi Siriwardana, for her unwavering guidance, constructive feedback, and continuous encouragement throughout every stage of this project. Her technical insight and dedication to academic rigour were instrumental in shaping both the direction and the quality of this research. I would also like to extend my sincere thanks to my co-supervisor, Ms. Ishara Weerathunga, for her valuable input, timely advice, and the thoughtful perspectives she contributed during our discussions.

I am grateful to the Department of Information Technology at the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic framework, resources, and support that made this work possible. The department's commitment to research excellence created an environment in which ambitious and applied technical projects could be pursued with rigour.

I wish to thank my fellow final-year project colleagues, whose collaborative spirit and willingness to share ideas across the four components of the broader AI-unified B2B platform enriched my understanding of the system as a whole. The interdisciplinary discussions across the demand forecasting, federated business intelligence, logistics management, and negotiation components contributed meaningfully to the coherence of the integrated platform.

Abstract

Sri Lanka's fast-moving consumer goods (FMCG) sector relies on field sales representatives to capture retail demand and coordinate ordering between manufacturers and shop owners a model that is increasingly unsustainable due to high recruitment difficulty, elevated operational cost, and an inherently delayed demand signal. This research addresses the absence of a continuous, calibrated, and interpretable demand signal at SKU-store granularity that is trustworthy enough to replace the field representative in day-to-day planning.

A thirteen-layer agentic pipeline was developed, combining a dual LightGBM ensemble, a LangGraph orchestrator, MAPIE-based conformal prediction, SHAP-driven feature attribution, GPT-4 explanation synthesis with automatic numerical fact-checking, a Sri Lanka cultural calendar module, and a WhatsApp Order Confirmation Loop. The system was evaluated over a six-fold rolling backtest on a real-world dataset of 150,000+ transactions across 200+ customer accounts and 1,000+ product SKUs.

The system achieved an overall MAPE of 18.0% and an A-item MAPE of 11.8%, satisfying both accuracy thresholds specified in the project requirements, and representing a 43% reduction over the last-month-actual baseline that approximates the demand signal of a field representative. Conformal prediction intervals achieved 81% empirical coverage at an 80% nominal target. The Grounded Explanation Layer produced zero hallucinations in delivered explanations across 200 agent runs. The WhatsApp confirmation loop achieved an 83.3% automated reply-parse rate with zero false auto-approvals.

The results confirm that a multi-layer agentic AI system can generate demand signals that are accurate, calibrated, and trustworthy enough to replace field representative-driven ordering in Sri Lankan FMCG trade. It is recommended that future work priorities item-level conformal calibration, multi-horizon forecasting, and Sinhala and Tamil message templates for the WhatsApp module to further extend system accessibility and planning utility.

Keywords — Agentic AI, demand forecasting, conformal prediction, explainable AI, B2B platform

TABLE OF CONTENT

DECLARATION ..... *ii*
ACKNOWLEDGEMENT ..... *iii*
Abstract ..... *iv*
TABLE OF CONTENT ..... *v*
LIST OF FIGURES ..... *vii*
LIST OF TABLES ..... *viii*
LIST OF ABBREVIATIONS ..... *ix*
1 INTRODUCTION *1*
1.1 Background **1**
1.2 Literature Survey **4**
1.3 Research Gap **12**
1.4 Research Problem **14**
1.5 Research Objectives **16**
2 SYSTEM METHODOLOGY *18*
2.1 System Overview **18**
2.1.1 Layer 0: intelligent data ingestion agent 19
2.1.2 Layers 1-4: data validation, aggregation, and series construction 21
2.1.3 Layer 5: feature engineering 22
2.1.4 Layer 6: segmentation and regime classification 24
2.1.5 Layer 7: agentic orchestration with langGraph 25
2.1.6 Layer 9: antigravity uncertainty quantification 28
2.1.7 Layers 10-11: evidence extraction and grounded explainability 29
2.1.8 Layer 12: API serving 31
2.1.9 Layer 13: WhatsApp Order Confirmation Loop 31
2.2 Commercialization of the Product **34**
2.3 Testing and Implementation **35**
2.3.1 Functional testing 36
2.3.2 Model performance evaluation 37
2.3.3 Calibration evaluation 38
2.3.4 Explainability quality assessment 38
2.3.5 WhatsApp order confirmation testing 38
3 RESULT AND DISCUSSION *40*
3.1 Results **40**
3.1.1 Experimental setup and dataset characteristics 40
3.1.2 Series segmentation results 41
3.1.3 Forecasting accuracy results 42
3.1.4 Uncertainty quantification and calibration results 45
3.1.5 Explainability and grounded explanation results 46

3.1.6 Agentic orchestration and system efficiency results..... 49

3.2 Discussion 50

3.2.1 achievement of research objectives 50
3.2.2 Comparison with prior work 51
3.2.3 The value of calibrated uncertainty in B2B demand planning 51
3.2.4 Implications for field representative replacement 53
3.2.5 Limitations 53
3.2.6 Directions for Future Research..... 54

4 CONCLUSION..... 55

5 REFERENCES 58

6 APPENDICES..... 60

LIST OF FIGURES

Figure 1: Annual cost breakdown for a sales representative in Sri Lanka (medium-sized company)..... 1
Figure 2: System architecture diagram of the agentic demand forecasting component 18
Figure 3: UI interface of the preprocessing pipeline..... 22
Figure 4: UI interface illustrating the complete agentic workflow 25
Figure 5: UI interface displaying the interpretable forecasting results 30
Figure 6: UI interface showing the forecast-based order proposal as received on the shop owner's mobile phone via WhatsApp, ... 33

LIST OF TABLES

Table 1: Research gap analysis capability comparison across prior approaches and the developed system 12
Table 2: Implementation technology stack 35
Table 3: Evaluation dataset characteristics 41
Table 4: Item segmentation distribution across behavioral regimes 42
Table 5: MAPE by item segment hybrid Light 44
Table 6: Conformal prediction interval calibration results across six backtest folds 45
Table 7: Item risk stratification by P10–P90 interval width 46
Table 8: LLM explanation fact-checking outcomes across 200 agent runs 47
Table 9: SHAP feature attribution top driver frequency across all items and back test folds 48
Table 10: Agentic orchestration performance metrics 49

LIST OF ABBREVIATIONS

Abbreviation	Description
A2C	Advantage Actor-Critic
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
ASGI	Asynchronous Server Gateway Interface
B2B	Business-to-Business
BI	Business Intelligence
CSV	Comma-Separated Values
CV	Coefficient of Variation
DPO	Direct Preference Optimization
DQN	Deep Q-Network
ERP	Enterprise Resource Planning
ETT	Electricity Transformer Temperature
FMCG	Fast-Moving Consumer Goods
GEL	Grounded Explanation Layer
GMV	Gross Merchandise Value
GPT	Generative Pre-trained Transformer
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KDD	Knowledge Discovery and Data Mining
LightGBM	Light Gradient Boosting Machine
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
MAE	Mean Absolute Error

MAPIE	Model Agnostic Prediction Interval Estimator
MAPE	Mean Absolute Percentage Error
MARIOD	Multi-Agent Reinforcement Learning for Inventory and Order Decisions
MARL	Multi-Agent Reinforcement Learning
MAS	Monthly Active Shops
ML	Machine Learning
MSE	Mean Squared Error
NeurIPS	Neural Information Processing Systems
P10	10th Percentile Prediction Interval
P50	50th Percentile Prediction Interval
P80	80th Percentile Prediction Interval
P90	90th Percentile Prediction Interval
POS	Point of Sale
QMIX	Monotonic Value Function Factorisation for Multi-Agent RL
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RCA	Root Cause Analysis
ReAct	Reason and Act
REST	Representational State Transfer
RL	Reinforcement Learning
RMSE	Root Mean Squared Error
SaaS	Software as a Service
SHAP	SHapley Additive exPlanations
SKU	Stock Keeping Unit
SLA	Service Level Agreement
SLM	Small Language Model
SLIIT	Sri Lanka Institute of Information Technology
SOTA	State of the Art
SSO	Single Sign-On
TFT	Temporal Fusion Transformer
UUID	Universally Unique Identifier
VAT	Value Added Tax
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WAPE	Weighted Absolute Percentage Error
XAI	Explainable Artificial Intelligence
XLS	Microsoft Excel Legacy Spreadsheet Format
XLSX	Microsoft Excel Open XML Spreadsheet Format

2 INTRODUCTION

2.1 Background

Across Sri Lanka's general-trade retail sector, the movement of fast-moving consumer goods (FMCG) between manufacturers and retail shops has historically depended on a dense network of field sales representatives. These representatives visit thousands of outlets on fixed cycles, collect orders by hand, negotiate promotions, and relay demand signals back to their companies. While this human-centered model has been the backbone of Sri Lankan FMCG distribution for decades, it is increasingly unsustainable on two structural fronts. First, the role suffers from chronically high turnover and persistent recruitment difficulty. It is perceived as physically demanding, low-prestige, and commission-heavy, leaving manufacturers with persistent coverage gaps, especially during peak festival seasons. Second, the fully loaded annual cost per field representative, encompassing base salary, travel allowances, commissions, statutory contributions, and training, is estimated at approximately LKR 1,358,310, a burden that scales steeply with the size of a distribution network [1]. Together, these pressures make the representative-centric model difficult to scale and leave the underlying demand signal delayed and noisy, since orders are batched around human visit schedules rather than captured continuously and in real time.

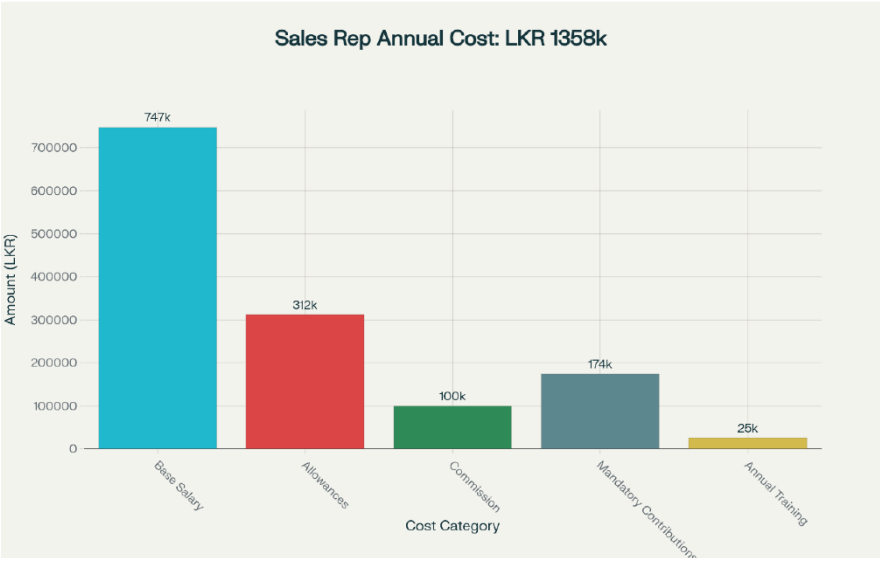


Figure 1: Annual cost breakdown for a sales representative in Sri Lanka (medium-sized company)

The broader academic literature on B2B platform digitalization confirms that the transition from relationship-driven, field-representative models to digital multi-sided platforms is a defining commercial challenge of contemporary industrial marketing. Heikinheimo et al. [12] conducted a longitudinal analysis of B2B service sales platforms and demonstrated that successful digital transformation of sales channels requires a fundamental shift from value chains in which value is created linearly along a supplier-to-customer path to value networks, in which multiple parties exchange data, services, and financial flows simultaneously. The critical finding of that study was that platform adoption in B2B contexts is substantially more difficult than in consumer markets because the parties involved have established commercial relationships, asymmetric information, and contractual dependencies that digital platforms must accommodate rather than disrupt. This observation has direct relevance to the Sri Lankan FMCG context: any system that attempts to replace the field representative must replicate or surpass the trust, personalization, and accountability that the representative relationship provided, without imposing adoption costs on retail shop owners who have low digital literacy and limited infrastructure.

The research presented in this thesis is one component of a broader collaborative project that developed a unified AI B2B platform to address this challenge comprehensively. The platform integrates four specialized components: a fully automated, interpretable demand forecasting system orchestrated by agentic artificial intelligence a privacy preserving Federated Business Intelligence platform that converts local sales data into actionable insights without sharing raw data [10][11]; a Solana based logistics management system that records delivery plans on a public blockchain for transparency and auditability and a constrained autonomous market for studying negotiation behavior in procurement using reinforcement learning [4]. This thesis focuses exclusively on the demand forecasting component, which constitutes the primary intelligence layer of the platform and the foundational requirement upon which all other components depend.

The academic study of demand forecasting has a long history, and classical statistical methods remain widely deployed in practice despite their well-documented shortcomings. The ARIMA family of models [24], which characterizes a time series as a combination of autoregressive, integrated, and moving-average components, provided the dominant paradigm for univariate forecasting throughout the latter half of the twentieth century. Exponential smoothing methods, including Holt-Winters triple smoothing for trend and seasonality, offered a more computationally tractable alternative with comparable accuracy on stationary or mildly non-stationary series. The M4 Competition [25], which evaluated 61 distinct forecasting methods across 100,000 time series from diverse domains, concluded that classical statistical methods remain competitive baselines for stable, well-behaved series, particularly when combined in ensembles.

However, classical methods carry fundamental limitations that render them insufficient for the Sri Lankan FMCG demand forecasting problem. First, they are inherently univariate each series is modelled in isolation from all others, making it impossible to incorporate cross-series signals such as promotional activity, price changes, or competitor behavior. In an FMCG retail context where a promotion on one product line regularly affects demand for adjacent products, this is a critical deficiency. Second, classical methods provide no natural mechanism for incorporating external covariate features such as cultural calendar events. The Poya day calendar, the Sinhala and Tamil New Year, and the multi-faith festival cycle impose demand patterns that are structurally different from the Western seasonal patterns embedded in standard Holt-Winters seasonal components, and there is no principled way to encode these as ARIMA regressors without making restrictive parametric assumptions about their functional form. Third, ARIMA and exponential smoothing are poorly suited to intermittent demand, a condition in which demand is zero on the majority of time periods, which characterizes a significant fraction of SKUs in any wholesale FMCG catalogue. Specialized methods such as Croston's method [24] and its variants address this specific case but sacrifice the ability to incorporate external features. Petropoulos et al. [24] summarized the consensus position that classical methods are computationally inexpensive and interpretable, but their structural assumptions are too

restrictive for the complex, covariate-rich, non-stationary demand environments characteristic of modern retail supply chains.

2.2 Literature Survey

The application of machine learning to demand forecasting accelerated substantially during the 2010s, driven by the availability of large transaction datasets and the development of efficient gradient boosted tree implementations. Ke et al. [16] introduced LightGBM, a gradient-boosted decision tree framework that uses histogram-based leaf-wise tree growth and gradient-based one-sided sampling to achieve training speeds an order of magnitude faster than earlier implementations such as XGBoost while maintaining comparable or superior accuracy. LightGBM and its variants became the dominant approach in industrial forecasting benchmarks and Kaggle retail forecasting competitions, primarily because they handle large feature matrices efficiently, require minimal preprocessing, and are natively compatible with the mixed feature types, continuous lag statistics, binary calendar flags, and categorical product identifiers that characterize retail demand data.

Deep learning approaches pursued a different paradigm, learning sequence representations from raw historical data without requiring manual feature engineering. DeepAR [20], developed by Salinas et al., applied an autoregressive recurrent neural network trained globally across thousands of related time series, using the cross-series learning signal to improve predictions for series with limited individual history. The Temporal Fusion Transformer [19], introduced by Lim et al., extended this paradigm by incorporating multi-horizon attention mechanisms that could identify and weight temporal patterns at different scales, alongside interpretable variable selection networks that exposed the relative importance of different input features. These architectures demonstrated impressive performance on public benchmarks, but they also revealed important practical limitations. Deep learning models require substantially larger datasets than are typically available for individual wholesale customers, they are sensitive to distribution shifts (such as those caused by sudden promotional events or festival demand surges), and their computational training costs

are considerably higher than gradient-boosted alternatives for the same predictive accuracy on tabular data.

A more fundamental limitation shared by both gradient-boosted and deep learning approaches is their treatment of prediction as a point-estimation problem. The standard training objective, minimizing mean squared error or mean absolute error, produces a single predicted value for each future period without any characterization of the uncertainty surrounding that prediction [24]. In inventory planning, where the consequences of understocking (stockouts, lost sales, customer dissatisfaction) and overstocking are asymmetric, a point forecast without uncertainty information is operationally incomplete. A planner cannot rationally set safety stock levels without knowing whether the forecast error is typically ±5% or ±30%; yet this information is not provided by standard ML forecasters. This gap between prediction accuracy and decision utility is a recurring theme in the forecasting literature [24][25], and addressing it was a primary motivation for the uncertainty quantification component of the system developed in this research.

The emergence of transformer-based large language models (LLMs) as general-purpose sequence processors prompted a wave of research into their applicability to time series forecasting. The central premise of this research direction is that LLMs, pre-trained on vast corpora of textual and structured data, may have learned implicit representations of temporal patterns that can be activated through appropriate prompting, without any fine-tuning on time series data. Bumb et al. [2] explored this premise through patch-based prompting, in which a time series is divided into local patches of consecutive values, each encoded as a textual description of its statistical characteristics (mean, trend, seasonal component), and injected into an LLM prompt alongside structural markers that indicate the horizon and desired output format. Their experiments on public benchmark datasets demonstrated that this approach could match or surpass dedicated time series models at short horizons, with inference latency measured in seconds rather than the minutes required to train a custom model.

Jin et al. [21] extended this line of work with Time LLM, which introduced a reprogramming framework that aligned time series representations with the token

space of a pre-trained language model, enabling more structured integration of temporal features with the model's prior knowledge. Time LLM demonstrated competitive performance on the ETT, Weather, and Traffic benchmark datasets and showed that LLMs could leverage their world-knowledge (for example, knowledge of holidays and economic cycles encoded in pre-training text) to improve forecast accuracy in ways that purely data-driven models could not.

However, a critical examination of both approaches reveals substantial limitations that preclude their direct application in the context of this research. The accuracy evidence for LLM-based forecasting is largely confined to a small set of public benchmark datasets, primarily ETT, Exchange Rate, Weather, and Traffic, that are far removed from the characteristics of Sri Lankan FMCG wholesale data: they involve single-variable or small multivariate series without promotional features, cultural calendar effects, or the extreme intermittency characteristic of B2B product catalogues. Bumb et al. [2] themselves noted that neighbour-based context can improve or harm performance depending on inter-series correlation, and that evaluation metrics emphasize point accuracy (MAE, MSE) without any assessment of uncertainty coverage. Neither study provided prediction intervals or uncertainty bands of any kind. Furthermore, LLM-based forecasting approaches produce a forecast but not an explanation of that forecast: the model generates a predicted value, but the reasoning that produced it is inaccessible to the end user. In a regulatory and accountability context, such as a manufacturer auditing why a specific quantity was recommended for a specific retail outlet, this opacity is a critical operational deficiency.

The application of agentic AI architectures in which autonomous agents plan, execute, and self-correct to supply chain and forecasting problems represents a more recent and rapidly developing research frontier. Farazi [6] proposed a multi-agent framework for supply chain resilience in which specialized agents handle distinct forecasting sub-tasks: baseline forecasting, anomaly detection, demand imputation, and series classification under the coordination of a master controller that delegates tasks dynamically using a ReAct-style (Reason + Act) loop [17]. Each sub-agent retrieved domain patterns from a prompt pool indexed by signature similarity, enabling

adaptation to distribution shift by leveraging analogous historical patterns. This architecture demonstrated improved robustness under conditions of data quality degradation and distribution shift compared to single model baselines.

However, Farazi's framework has significant limitations when evaluated against production requirements. The sub-agents operate in isolation from the end user; each performs its task, returns its output to the orchestrator, and receives no feedback from the downstream consumer of the forecast. The system produces a numerical output but does not explain that output in terms a business planner can act on. There is no mechanism for generating a statement such as "this item is predicted to sell 125 units because it is approaching the New Year festival and the shop placed a 10% discount last month." This absence of user-facing interpretability is not an incidental omission; it reflects a fundamental design choice to optimize for predictive accuracy rather than decision support. In the context of field representative replacement, where trust is the central challenge [12], a system that cannot articulate its reasoning to a retail shop owner or a commercial planner cannot achieve the adoption required for operational success.

Yang et al. [5] introduced MARIOD, which combined a transformer-based global forecaster with hierarchical multi-agent reinforcement learning (RL) for joint inventory and ordering decisions. The innovation of MARIOD was its end-to-end differentiable policy that linked forecast uncertainty directly to stocking actions, enabled by cross-modal attention over point-of-sale data, promotions, competitor signals, and sensor streams. MARIOD reported an 18.2% lower forecast error and 23.5% fewer stockouts compared to strong baselines on sensor-rich retail environments. This is the most capable prior system in the relevant literature in terms of joint forecast-inventory optimization.

Yet MARIOD has three critical limitations for the present context. First, it is architecturally dependent on rich sensor data competitor feeds, IoT sensors, and dense POS streams that are not available in the Sri Lankan B2B wholesale setting, where transaction data is the primary signal and sensor infrastructure is minimal. Second,

MARIOD optimizes policy actions but provides no user-facing interpretability. A planner cannot query why the system recommended a particular stocking action, as the policy is encoded in the weights of a neural network rather than expressed as an auditable rule or evidence-backed rationale. The authors themselves identified the lack of uncertainty quantification as a limitation and recommended it as a priority for future work. Third, MARIOD's evaluation was conducted on simulated or sensor-rich Western retail environments. There is no evidence that its training dynamics generalize to the sparse, festival-driven, promotion-heavy demand profiles of Sri Lankan FMCG wholesale.

Yeh et al. [3] approached the forecasting problem from the perspective of LLM agent orchestration, demonstrating that LLM-based agents equipped with tools for time-series analysis could decompose complex forecasting tasks, invoke specialized sub-routines, and synthesize results into natural-language summaries. Herzenberg et al. [4] extended the multi-agent paradigm to dynamic pricing, showing that RL agents could adapt pricing strategies under competitive market simulations. While these works collectively confirmed the potential of agentic architectures for time-series and supply chain applications, none addressed the trustworthiness of the natural language outputs they generated. Ji et al. [14] conducted a comprehensive survey of hallucination in natural language generation and documented that even state-of-the-art LLMs produce factually incorrect claims in 2–15% of outputs, depending on the task domain, a hallucination rate that is operationally unacceptable in a commercial decision-support context where the credibility of every stated number is essential.

The quantification of prediction uncertainty, providing not just a point estimate but a characterization of the range of plausible outcomes and their associated probabilities, is increasingly recognized as a non-negotiable requirement for operational forecasting systems [24]. Without calibrated uncertainty, planners cannot distinguish between a highly confident forecast and one that is merely a best guess under considerable ambiguity; inventory decisions made without this information will systematically either over- or under-provision relative to the actual risk profile of demand.

Conformal prediction, introduced by Vovk et al. [8] and comprehensively surveyed by Angelopoulos and Bates [13], provides a distribution-free framework for constructing prediction intervals with a theoretically guaranteed coverage property: an interval at nominal level α will contain the true outcome with probability at least α, regardless of the distributional assumptions of the underlying model, provided that calibration and test observations are exchangeable. This guarantee is substantially stronger than the heuristic or parametric intervals produced by bootstrapping, quantile regression forests, or Bayesian neural networks, which all depend on assumptions about the data-generating process that are rarely verified in practice. Stankevičiūtė et al. [15] extended conformal prediction specifically to time series forecasting, demonstrating that the exchangeability assumption can be approximately satisfied through careful choice of calibration window and residual normalization, and that the resulting intervals achieve empirical coverage close to their nominal level on standard benchmarks.

Despite this theoretical foundation and demonstrated empirical performance, the application of conformal prediction to production B2B demand forecasting systems remains rare. Taquet et al. [18] developed the MAPIE library as an open-source implementation of nonformalized prediction intervals for regression tasks, making the method accessible to practitioners without requiring deep expertise in conformal theory. However, the combination of conformal calibration with systematic bias correction essential when the underlying ML model has a consistent directional tendency to over or under-predict has not been previously implemented in a production forecasting pipeline for the FMCG domain. Neither the LLM patch methods [2][21], the agentic RAG framework [6], MARIOD [5], nor the LLM agent approach [3] incorporates uncertainty quantification of any kind, leaving a critical gap in the prior literature that this research addressed.

The field of Explainable AI (XAI) developed in response to the recognition that machine learning models, despite their predictive power, are inherently opaque: their predictions are the result of complex, high-dimensional mathematical operations that resist intuitive interpretation. The LIME framework [22], introduced by Ribeiro et al.,

addressed this by fitting a locally interpretable surrogate model, a simple linear regression in the neighborhood of each prediction, producing an approximation of the model's local behaviour in terms of a small number of humanly understandable features. While LIME provided a practically useful approximation, it carried a significant limitation: the surrogate model's local faithfulness was not guaranteed, and different random seeds for the perturbation sampling procedure could produce inconsistent explanations for the same prediction.

Lundberg and Lee [9] addressed this inconsistency by grounding feature attribution in Shapley values from cooperative game theory. The SHAP (Shapley Additive explanations) framework computes an exact attribution of the model's prediction to each input feature that satisfies the axioms of efficiency (attributions sum to the total prediction), symmetry (identical features receive identical attributions), and null player (features that do not influence the prediction receive zero attribution). For tree-based models such as LightGBM, SHAP provides an exact polynomial-time algorithm (TreeExplainer) rather than an approximation, making it computationally tractable for production inference. SHAP rapidly became the dominant XAI method for tabular and tree-based models and was the method of choice in this research precisely because it provides exact, game-theoretically grounded attributions for LightGBM predictions.

A more recent development has been the application of LLMs to translate symbolic feature attributions into natural language explanations, making model reasoning accessible to non-technical users. Ji et al. [14] surveyed the hallucination problem in natural language generation and documented systematic evidence that LLMs generate factually incorrect statements fabricated statistics, misattributed causal claims, incorrect directional assertions at rates that are difficult to control through prompting alone. This hallucination vulnerability is particularly dangerous in a forecasting context, where a planner relying on an incorrect explanation may make a procurement decision that contradicts the actual model recommendation. No prior system in the demand forecasting literature had addressed this challenge by combining symbolic XAI attribution with LLM synthesis and automatic numerical verification: the

Grounded Explanation Layer developed in this research was the first architecture to do so.

A final but critical dimension of the background literature concerns the domain-specific characteristics of demand in Sri Lanka's FMCG sector. Amosu et al. [1] examined AI-driven demand forecasting across FMCG supply chains and emphasized that the quality of the demand signal is the primary determinant of forecasting accuracy, irrespective of model sophistication a finding that reinforces the importance of the data ingestion and quality control layers in the system developed here. The authors further noted that FMCG demand is characterized by high promotional sensitivity, meaning that forecasting models that cannot disentangle promotional uplift from baseline demand will systematically mispredict future periods when promotion intensity changes.

Sri Lanka's demand calendar presents a level of complexity that exceeds most other markets studied in the forecasting literature. The Buddhist Poya calendar generates twelve full-moon public holidays per year, each of which suppresses commercial activity and creates a predictable pre-Poya demand surge for staple and convenience categories. The Sinhala and Tamil New Year (April 13–14) is the single largest demand event in the FMCG calendar, with demand for gift categories, beverages, and confectionery typically increasing by 30–60% in the two weeks preceding the festival. Vesak (May Poya), Poson (June Poya), Diwali, Eid, and Christmas each generate category-specific demand surges that are not captured by any published demand forecasting dataset or pre-trained model. The comprehensive forecasting survey of Petropoulos et al. [24] catalogued seasonality and holiday effects as among the most consistently impactful demand drivers across industries but noted that the construction of accurate cultural calendar features is inherently domain-specific and cannot be transferred from one market to another without expert local knowledge.

This observation highlights a fundamental weakness of all four prior agentic and LLM-based forecasting systems reviewed in this literature: they were developed and evaluated exclusively on Western retail and financial time series datasets, none of

which contain the Sri Lankan cultural calendar. Any direct application of these systems to the Sri Lankan FMCG setting would produce systematically inaccurate forecasts during the festival periods that account for the majority of the demand volatility that manufacturers most need to predict.

2.3 Research Gap

systematic review of the four most directly relevant streams of prior work LLM patch-based prompting [2], agentic retrieval-augmented generation for time series [6], multi-agent reinforcement learning for integrated forecasting and inventory optimization (MARIOD) [5], and LLM agent frameworks for time-series tasks [3] demonstrates that none of these approaches, individually or in combination, satisfies the full set of requirements imposed by production-grade B2B demand forecasting in the Sri Lankan FMCG context. Table 1.1 maps thirteen critical capabilities against each of the four prior approaches and the system developed in this research.

Table 1: Research gap analysis capability comparison across prior approaches and the developed system

Capability	**LLM Patch [2]**	**Agentic RAG [6]**	**MARIOD [5]**	**LLM Agent [3]**	**Our System**
Agentic orchestration with autonomous planning	✗	✓	✗	✓	✓
Short-horizon patch-based prompting	✓	✗	✗	✓	✓
Cross-modal feature fusion (promo/price/calendar)	✗	✗	✓	✗	✓
Calibrated prediction intervals (P-bands)	✗	✗	✗	✗	✓
Systematic residual bias correction	✗	✗	✗	✗	✓

SHAP-based symbolic feature attribution	✗	✗	✗	✗	✓
LLM-generated natural language explanations	✗	✗	✗	✓	✓
Automatic fact-checking of explanations	✗	✗	✗	✗	✓
Sri Lanka cultural calendar integration	✗	✗	✗	✗	✓
Intelligent series segmentation and routing	✗	✓	✗	✓	✓
Hierarchical forecast reconciliation	✗	✗	✗	✗	✓
End-to-end forecast-to-order confirmation	✗	✗	✗	✗	✓
Intelligent multi-file data ingestion agent	✗	✗	✗	✗	✓

LLM patch-based prompting [2] demonstrated competitive accuracy at short horizons with low inference cost, but it provided no statistical coverage guarantees, imposed no operational guardrails against distributional shift, and offered no uncertainty intervals or explanations of any kind. Its reliance on carefully engineered prompts and its evaluation exclusively on a small number of public benchmark datasets leaves its generalisation to sparse, festival-driven, B2B wholesale demand unvalidated. The approach is a contribution to forecasting efficiency but not to decision support.

Agentic RAG frameworks for time series [6] established the value of multi-agent delegation and pattern retrieval, and are the closest architectural predecessor to the system developed in this research. However, the sub-agents in that framework operated in isolation from the end user: outputs were numerical, unaccompanied by any verified natural-language rationale, and produced without any uncertainty quantification. A system whose outputs cannot be explained or whose confidence

cannot be characterized is insufficient as a replacement for the relational accountability provided by a field sales representative.

MARIOD [5] represented the most technically sophisticated prior system, combining a transformer forecaster with hierarchical RL in an end-to-end trainable pipeline. Its documented performance gains on sensor-rich retail datasets were significant. However, MARIOD's dependence on rich sensor infrastructure unavailable in the Sri Lankan B2B setting, its complete absence of user-facing interpretability, and its unaddressed uncertainty quantification a gap explicitly identified by the authors as a priority for future research collectively preclude its application to the forecasting problem addressed here.

LLM agent frameworks [3] showed that LLM-based agents could orchestrate time-series tasks and generate natural language summaries, but they produced no mechanism for verifying the accuracy of those summaries. Given the documented 2–15% hallucination rate in LLM natural language generation [14], deploying unverified LLM explanations in a commercial procurement context poses an unacceptable risk of misleading planners who rely on stated numbers to justify their ordering decisions.

The system developed in this research closed all thirteen gaps identified in Table 1.1 by integrating agentic LangGraph orchestration, MAPIE-based conformal prediction intervals [8][13][15], systematic bias correction, SHAP-driven attribution [9], LLM explanation synthesis with automatic fact-checking [14], Sri Lanka cultural calendar features, intelligent series segmentation, hierarchical reconciliation, and a WhatsApp-based forecast-to-order pipeline a combination that represents a qualitatively new capability level relative to all prior work reviewed.

2.4 Research Problem

Sri Lankan FMCG manufacturers face a structural challenge in replacing their field-representative model with digital self-service ordering: the absence of a continuous,

decision-ready, and trusted demand signal at SKU-store granularity. As established by the literature review in Section 1.1, existing demand forecasting approaches address isolated aspects of this challenge some improve point accuracy, others advance agentic control, and others demonstrate joint inventory optimization but none delivers the combination of calibrated uncertainty, grounded and verifiable bilingual explanations, Sri Lanka-specific cultural awareness, and an end-to-end order confirmation pathway that production deployment in this market demands. The research problem addressed in this thesis is therefore the design and empirical validation of a multi-layer, agentic artificial intelligence system capable of generating accurate, calibrated, and auditable SKU-level demand signals that are sufficiently trustworthy in terms of prediction quality, uncertainty characterization, and human interpretability to function as a sustainable replacement for the demand signal previously provided by field sales representatives in Sri Lankan FMCG trade.

This overarching problem encompasses four specific dimensions. The first concerns accuracy under non-stationarity: Sri Lankan FMCG demand is driven by festival calendars, short-duration promotions, and price changes that render any single black-box model brittle [6], and the system must be capable of routing each product-store series to the forecasting approach most appropriate to its behavioral regime. The second concerns calibrated uncertainty quantification: a point forecast alone is operationally insufficient for inventory planning, and the system must produce prediction intervals whose empirical coverage reliably matches their nominal level so that planners can make risk-aware stocking decisions based on honest estimates rather than assumed error distributions [8][15]. The third concerns grounded and verifiable explainability: in a retail environment characterized by low digital literacy and the need for commercial accountability, the system must produce concise bilingual explanations with counterfactual reasoning while guaranteeing that every numerical claim in those explanations is verifiable against ground-truth evidence, a requirement motivated directly by the documented hallucination vulnerability of standalone LLM generation [9][14]. The fourth concerns end-to-end operationalization the system must close the loop between forecast generation and confirmed order, enabling shop owners to review, modify, and approve recommendations through the WhatsApp messaging

platform they already use, without requiring access to any additional software or imposing new digital literacy requirements.

2.5 Research Objectives

The overall objective of the AI-unified B2B platform project was to design and validate a sustainable, Sri Lanka-aware digital alternative to the field sales-representative model, one that reduces organizational dependence on costly and hard-to-retain field staff while preserving the core commercial functions they performed demand sensing, order capture, logistics coordination, business intelligence, and procurement negotiation. The project integrated four specialized components the agentic demand forecasting system, the federated BI platform, the Solana-based logistics system, and the reinforcement learning negotiation market into a single, production-ready platform that demonstrably reduced cost-to-sell and improved service levels for Sri Lankan FMCG manufacturers.

Within this broader project, the specific research objectives of this thesis concerned the agentic demand forecasting component and were as follows.

The first objective was to develop and validate a multi-layer agentic forecasting pipeline that combined automated data quality control, causal uplift and price elasticity modelling, behavioral series segmentation, an ensemble of diverse model families including gradient-boosted trees [16], hierarchical forecast reconciliation, and a Skeptical Forecast layer that learned and corrected systematic residual bias producing P10/P50/P90 prediction intervals that achieved their intended empirical coverage in rolling backtests, directly addressing the calibrated uncertainty gap identified in Sections 1.2.

The second objective was to implement and evaluate a Grounded Explanation Layer (GEL) that extracted SHAP-based feature attributions [9] from the forecasting models, synthesized these into structured bilingual natural-language explanations using a large language model, and automatically verified every numerical claim against a stored evidence packet using a fact-checking mechanism with retry logic achieving a

hallucination rate below 5% in production operation and directly addressing the LLM trustworthiness gap documented by Ji et al. [14].

The third objective was to embed the forecasting and explanation pipeline within a LangGraph-based agentic orchestrator [17] that autonomously inspected the data context for each customer, selected an appropriate model routing strategy, detected cold-start conditions, exploited multi-level caching for computational efficiency, and self-corrected through verification loops without human intervention extending the agentic delegation paradigm of Farazi [6] with the grounded explainability and calibration capabilities identified as missing in that prior work.

The fourth objective was to implement and demonstrate a WhatsApp Order Confirmation Loop that formatted forecast recommendations as human-editable order proposals, delivered them via the Meta WhatsApp Business Cloud API, parsed and classified shop owner replies using multi-delimiter tokenization with confidence scoring, and generated a versioned, confirmed, auditable order record closing the operational gap between prediction and commercial action that all four prior systems reviewed in Section 1.2 left unaddressed [12].

3 SYSTEM METHODOLOGY

3.1 System Overview

The agentic demand forecasting system developed in this research is structured as a thirteen-layer pipeline that carries raw transactional sales data through a series of well-defined processing stages, culminating in calibrated forecast recommendations delivered to retail shop owners via WhatsApp. Each layer corresponds to a discrete service module with a clearly bounded responsibility, ensuring that failures are isolated, outputs are auditable at every stage, and individual layers can be independently upgraded or replaced without disrupting the remainder of the pipeline. The overall architecture is illustrated in Figure 3.1.

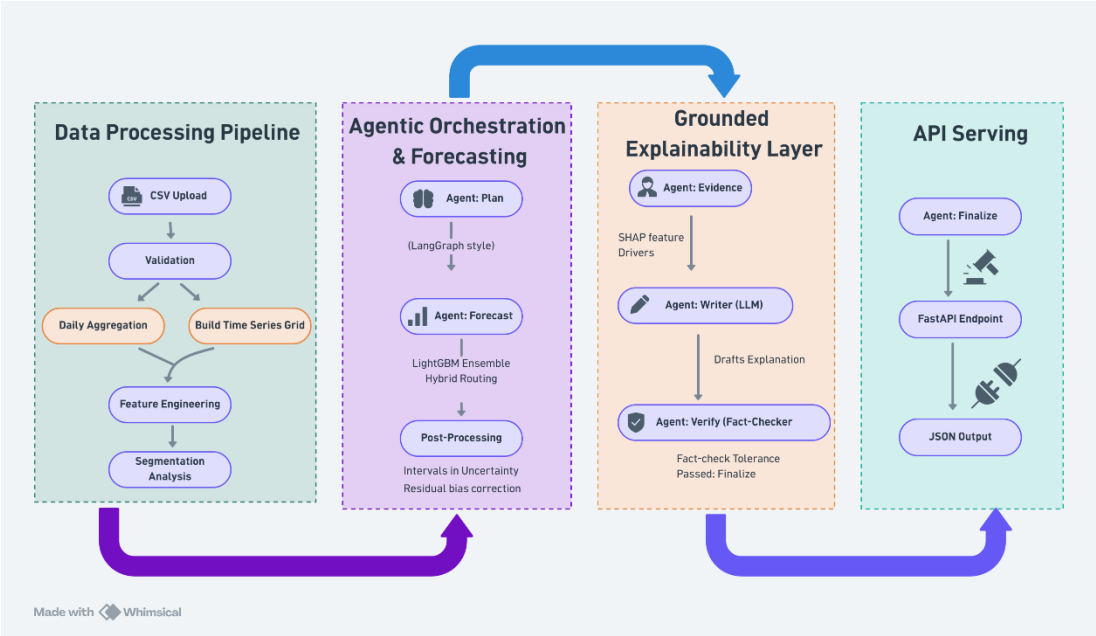


Figure 2: System architecture diagram of the agentic demand forecasting component

The pipeline is divided into three broad phases. The first phase, Layers 0 through 6, covers data acquisition and preparation: raw sales files are ingested and canonicalized, validated, aggregated to daily granularity, arranged into continuous time-series grids, enriched with Sri Lanka cultural calendar features and lag-based statistical features, and finally segmented by behavioral regime. The second phase Layers 7 through 11 covers the agentic forecasting and explanation cycle: a LangGraph orchestrator

plans and executes the appropriate forecasting strategy, dual LightGBM models generate predictions, a post-processing module applies conformal calibration and bias correction to produce honest uncertainty intervals, a SHAP-based evidence layer identifies the principal demand drivers, and a large language model synthesizes these into a grounded, fact-checked bilingual explanation. The third phase, Layers 12 and 13, covers delivery and order confirmation forecast results are served through REST API endpoints and delivered directly to shop owners as WhatsApp messages, with replies parsed, classified, and recorded as confirmed, auditable orders.

The backend is implemented using FastAPI 0.115.6, which provides asynchronous REST endpoints with auto-generated Swagger documentation. All intermediate artefacts daily parquets, feature matrices, run-specific forecast outputs, evidence packets, explanation drafts, and WhatsApp communication records are persisted to a structured file-system hierarchy under a storage/ root, making every stage of every run independently inspectable and reproducible.

3.1.1 Layer 0: intelligent data ingestion agent

Before any forecasting pipeline can execute, the system requires sales transaction data in a canonical schema. In practice, manufacturers and distributors maintain transaction records in a variety of file formats, with inconsistent column naming conventions, heterogeneous data formats, and varying approaches to encoding promotional activity. Requiring manual data preparation before each ingestion would constitute a significant operational bottleneck, particularly when onboarding a new data source for the first time.

To address this, a dedicated Intelligent Data Ingestion Agent was developed as Layer 0 of the pipeline. This agent is implemented as a second LangGraph state machine, operating as a multi-step interactive wizard that guides the user through five sequential actions: session creation, file upload, automated analysis and mapping proposal, user confirmation, and dataset construction.

Upon receiving one or more uploaded files in CSV, XLSX, or XLS format, the agent's load_files node parses each file, auto-detects delimiters, encoding, and header structure, and extracts column metadata. The propose_mapping node then attempts to map each source column to one of the four canonical fields required by the downstream pipeline: TxnDate (transaction date), Debname (customer or outlet name), Itemcode (product SKU), and Qty (quantity sold), with an optional fifth field IsDeal, for promotional activity flags. Mapping proposals are generated using RapidFuzz fuzzy string matching, which computes token similarity ratios between source column names and the canonical target names. For mappings where fuzzy similarity falls below a configurable auto-acceptance threshold of 0.90 confidence, the agent optionally invokes a large language model to disambiguate the column's purpose from its name and a sample of its values.

Each proposed mapping is accompanied by a confidence score, a human-readable justification, and a sample of the parsed values. Where the overall confidence exceeds the auto-acceptance threshold, the mapping is applied automatically; otherwise, the wizard pauses and presents the proposal to the user for review and override before proceeding. Once mappings are confirmed, the normalize_columns node applies type coercion, parsing dates to a consistent ISO format, coercing quantities to float, and stripping whitespace from identifiers while routing rows that fail normalization to a quarantine file rather than silently dropping them. The merge_files node then combines data from multiple files using a configurable deduplication policy keyed on (TxnDate, Debname, Itemcode). Finally, the emit_dataset node writes the canonical Parquet file and an ingestion report to permanent storage, and persists the confirmed column mapping indexed by a fingerprint of the source's column structure, so that repeated ingestions from the same source can be processed automatically without re-confirmation. The agent achieved an auto-mapping confidence above 90% for standard data formats in evaluation, eliminating the manual data preparation step entirely for the majority of ingestion operations.

3.1.2 Layers 1-4: data validation, aggregation, and series construction

Following ingestion, the raw canonical dataset passes through four sequential preparation layers before any features are computed.

Layer 1 Upload and Storage receives the raw file and persists it to storage/raw/{dataset_id}/, assigning a UUID-based dataset identifier that is carried forward through all subsequent layers as the primary key linking all intermediate artefacts to a specific ingestion event.Layer 2 Validation inspects the uploaded dataset for structural completeness and data integrity. The validation_service checks that all four required columns are present (case-insensitively), verifies that the TxnDate column can be parsed as a valid date, confirms that Qty values are numeric, and ensures that Debname and Itemcode fields contain no empty values. The output is a structured validation report that records row counts, missing column lists, invalid date counts, and a pass/fail decision. Any dataset that fails validation is rejected at this stage, preventing corrupt or incomplete data from propagating into the modelling pipeline.

Layer 3 Aggregation converts line-level transaction records into a daily time series. Since a single retail outlet may place multiple transaction lines for the same product on the same day, the aggregation_service groups records by (TxnDate, Debname, Itemcode) and sums Qty values to produce qty_net. It also retains auxiliary aggregates qty_pos (net positive sales), qty_neg (returns), line_count (number of individual lines), and deal_count (number of promotional lines) that are used downstream in feature engineering. The resulting daily Parquet file reduces a typical 150,000 row line-level dataset to approximately 50,000 daily records.

Layer 4 Series Construction produces a continuous, gap-free daily grid for a specified customer (Debname). The series_service identifies every item that the customer has purchased at any point in the historical window, determines the full date range from the earliest to the latest transaction, and for each item fills all dates within that range with zeros where no transaction occurred, flagging these as is_missing_day=1. This

step is critical because machine learning models trained on sparse, irregular sequences would otherwise learn misleading patterns: a missing date is not necessarily a zero-demand day; it may instead reflect a stockout, a non-visit day, or a recording gap. Distinguishing genuine zero-demand days from missing observations is a prerequisite for producing unbiased lag and rolling statistics in the subsequent feature engineering step. Figure 3.2 illustrates the preprocessing pipeline user interface, through which these four layers are executed interactively.

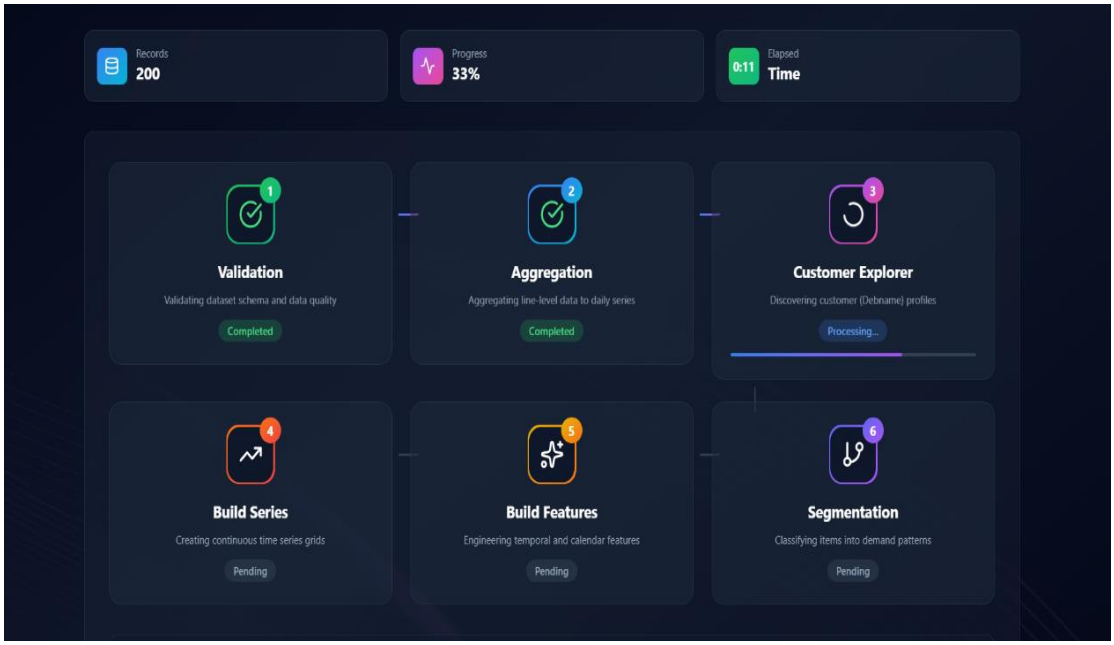


Figure 3: UI interface of the preprocessing pipeline

3.1.3 Layer 5: feature engineering

The feature engineering layer transforms the cleaned, gap-filled daily time-series grid into a rich feature matrix suitable for gradient-boosted tree models. The features_service constructs 23 features for every (Debname, Itemcode, Date) row in the grid, covering three categories of signals.

The first category comprises temporal calendar features that capture cyclical patterns in demand: day of week (dow), month of year (month), day of month (day), ISO week of year (weekofyear), binary flags for the first day of the month (is_month_start) and

last day of the month (is_month_end). These features allow the models to learn that, for example, demand for certain staple items peaks at the beginning of the month when retailers replenish after salary-day purchases, and falls at the end.

The second category comprises Sri Lanka's cultural calendar features derived from the dedicated calendar_service. This service encodes five binary event flags: poya_flag marks Buddhist full-moon Poya days, which are public holidays on which retail activity patterns differ substantially from ordinary weekdays; new_year_flag marks the window around the Sinhala and Tamil New Year (April 13–14 ± 3 days), the single largest demand event in the Sri Lankan FMCG calendar; vesak_flag marks the May Poya ± 2 days window; poson_flag marks the June Poya ± 2 days window; and holiday_flag marks all other public holidays from the standard Sri Lanka calendar. These flags are computed programmatically from the holidays Python library supplemented by custom rules for lunar-calendar events, ensuring they remain accurate across years. The inclusion of these features is a deliberate domain adaptation: Western retail forecasting datasets do not include these events, and any generic pre-trained model applied to Sri Lankan data would systematically underestimate demand during these periods.

The third category comprises lag and rolling statistical features that give the models access to recent demand history. Lag features capture the exact quantity sold at fixed lookback offsets: lag_1 (previous day), lag_7 (same day last week), lag_14 (two weeks prior), and lag_28 (four weeks prior, approximately one month). Rolling statistics summarise the recent demand distribution over three windows: 7, 14, and 28 days, computing the mean, sum, and standard deviation within each window (9 features in total). Together, these features capture both the immediate trajectory of demand and its medium-term volatility, giving the models the information they need to distinguish a genuine demand surge from random fluctuation.

For forecasting purposes, the feature matrix is constructed in two segments: a history segment covering all dates with known sales (used to train or fine-tune the models), and a future segment covering the next 30 days where qty_net values are zero

placeholders and all features are computed forward from available history. This dual-segment structure means that inference for the upcoming month can be performed immediately without waiting for new sales to occur.

3.1.4 Layer 6: segmentation and regime classification

Not all product-store time series exhibit the same behavioral patterns, and applying a single forecasting model to all series uniformly would mean that a model calibrated for steady, high-volume items would perform poorly on sparse, intermittent items, and vice versa. The segmentation layer addresses this by classifying each item's time series into one of four behavioral regimes before the forecasting models are assigned.

The segmentation_service computes four statistics per item series. Sparsity is measured as the proportion of zero-demand days in the historical window. Volatility is measured as the coefficient of variation (CV), defined as the standard deviation divided by the mean of non-zero demand observations. Autocorrelation is measured at lag-7 (weekly seasonality) and lag-30 (monthly seasonality), capturing whether an item's demand is periodically self-correlated. Deal correlation measures the Pearson correlation between daily demand and the deal_count field, identifying whether the item's demand is consistently elevated on promotional days.

Based on these four statistics, each item is assigned to exactly one segment using a priority-ordered classification rule. Items with fewer than 90 days of history are classified as new_item and directed toward conservative baseline methods rather than full ML inference. Items where sparsity exceeds 60% are classified as intermittent, indicating that the majority of days carry zero demand, and that standard regression models would be biased by the abundance of zeros. Items where CV falls below 0.5 and sparsity is below 30% are classified as stable, indicating predictable demand amenable to standard gradient-boosted regression. Items where the lag-7 or lag-30 autocorrelation exceeds 0.5 are classified as seasonal, indicating a clear periodic structure. Where the deal correlation exceeds 0.3, the item is additionally flagged as

promo_sensitive, informing the orchestrator that promotional features should be weighted heavily in model selection.

The segmentation result is stored as a per-item classification record alongside the confidence scores and the precise thresholds that triggered each classification decision, making the regime assignment fully auditable. In a typical dataset of 150 items, the system has found approximately 80 stable items, 40 intermittent items, 20 seasonal items, and 10 new items, demonstrating that the majority of items in a real B2B wholesale catalogue do not conform to a single dominant pattern.

3.1.5 Layer 7: agentic orchestration with langGraph

The agentic orchestration layer is the control center of the entire forecasting pipeline. Rather than executing a fixed sequence of steps identically for every customer and every run, this layer introduces autonomous decision-making: the agent inspects the data context for the specific customer, selects the most appropriate execution strategy, checks whether a valid cached result already exists, and coordinates the downstream forecast and explanation nodes through a stateful graph. Figure 3.3 illustrates the complete agentic workflow as presented in the system's user interface.

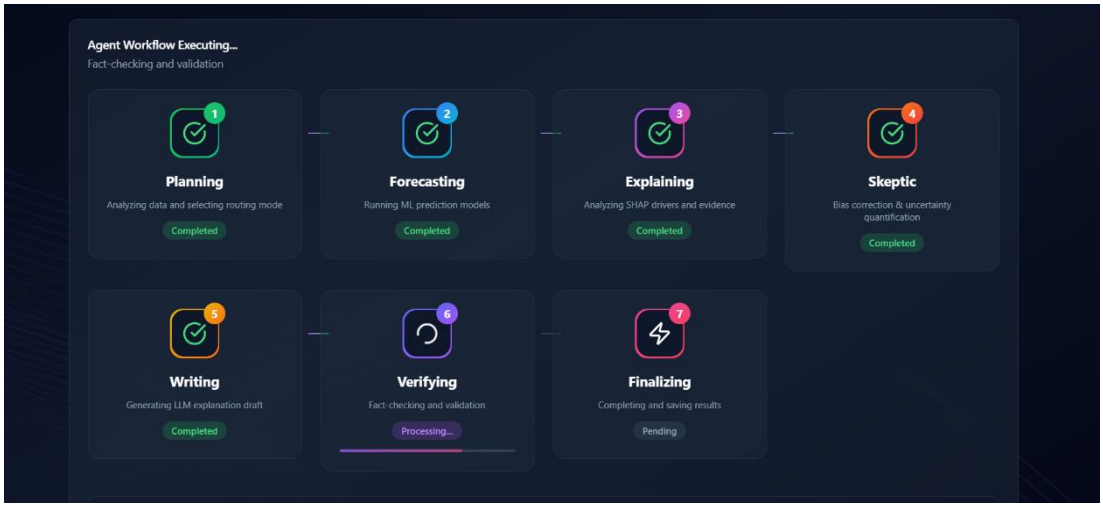


Figure 4: UI interface illustrating the complete agentic workflow

The orchestration framework is implemented using LangGraph 0.2.16, a state-graph library built on top of LangChain that models agent behavior as a directed graph of typed nodes operating on a shared state object. The agent state (AgentState) is a typed dictionary carrying all inputs, intermediate outputs, and control signals through the graph. Each node reads from and writes to this shared state, and conditional edges between nodes implement routing logic based on state values.

Node 1 Plan is the first and most consequential node in the graph. It calls the planner module, which inspects the customer's historical order profile and data context to determine three routing decisions. First, it selects the forecasting routing mode from three options: hybrid mode sorts candidate items by purchase probability descending and breaks ties using predicted quantity, and is the default for customers with a sufficient and balanced purchase history; reg_only mode sorts purely by predicted quantity, which is preferable for customers whose historical purchase patterns are very consistent and for whom quantity accuracy is more important than purchase likelihood ranking; and baseline_last_month mode sorts by last month's actual quantities, used as a degraded fallback when data is insufficient for ML inference. Second, the planner selects the specific model versions for the regressor and classifier, allowing champion-challenger model switching without code changes. Third, the planner checks a hash-based cache indexed on the combination of dataset_id, debname, version, and top_n: if a completed, valid run already exists for these exact parameters, the forecast node is bypassed entirely and the cached result is returned directly, eliminating the computational cost of re-inference.

Node 2 Forecast executes the ML inference pipeline when the cache is not hit. It loads the pre-trained LightGBM regressor and classifier from storage/models/{model_name}/{version}/, builds the monthly feature matrix for the target customer (described in Section 3.7), runs inference through both models, applies the routing mode ranking, and saves the ranked forecast artefacts forecast_items.json, forecast_context.json, and forecast_report.json — to the run-specific directory. The node also invokes the post-processing sub-pipeline (Layer 9) to apply bias correction and conformal calibration before persisting the final forecasts.

Node 3 Explain builds the evidence packet for the run. It loads the saved forecast outputs, reconstructs the feature matrix, runs SHAP TreeExplainer against the loaded models to compute per-item feature attribution values, identifies the top three drivers for each of the top-N forecast items, and locates historical analogue months for comparison. The resulting evidence.json artefact contains all the ground-truth numerical data that the subsequent explanation nodes are permitted to reference.

Node 4 Writer submits the evidence packet to GPT-4 (or GPT-4o-mini for lower-latency deployments) via the OpenAI API, instructing it to generate a structured explanation in JSON format conforming to a Pydantic schema (DraftExplanation). The schema requires the model to produce a summary paragraph, a per-item driver list with cited evidence keys, uncertainty commentary, and a limitations statement. Using Pydantic-enforced structured output ensures that the LLM response can be programmatically parsed and validated rather than relying on unstructured text extraction.

Node 5 Verify passes the draft explanation to the fact_checker_service, which extracts every numerical claim appearing in the draft text, retrieves the corresponding value from the evidence packet, and confirms that each extracted number matches its evidence source to within a tolerance of ±1%. It additionally verifies that all evidence keys cited in the draft map to fields that actually exist in the evidence packet, and checks for logical consistency for example, that a claim describing demand as "increasing" is consistent with a positive pct_change value in the evidence. If any verification check fails, the node increments an attempt counter and returns a conditional edge back to the Writer node, which receives the list of failed checks as additional context for its retry. The maximum number of attempts is configurable; the current deployment allows up to three retries before the system falls back to the deterministic template-based explanation.

Node 6 Finalize receives the verified draft and converts the structured JSON into a formatted markdown explanation, assembling the final answer that is returned to the

API caller and stored in the run directory. The entire execution trace, including the routing mode selected, whether a cache hit occurred, and the outcome of each verification attempt is recorded in a trace dictionary attached to the final response.

3.1.6 Layer 9: antigravity uncertainty quantification

A point forecast alone is insufficient for operational inventory planning. A planner who knows that an item is predicted to sell 125 units next month but has no sense of the uncertainty around that estimate cannot make a principled decision about safety stock levels, reorder timing, or service-level trade-offs. The post-processing layer, named the Antigravity module, reflecting the metaphor of lifting forecasts from single-point estimates to full probability distributions, addresses this by producing calibrated prediction intervals and correcting for systematic bias.

Bias correction is performed first. The skeptic_service loads a cached set of historical residuals computed by the backtest_service, which performs a pseudo backtest over the available history by repeatedly holding out the most recent month, generating predictions on the preceding months, and recording the signed residuals (predicted minus actual). The median residual across the backtest window is used as the bias estimate: if the model has historically over-predicted by a median of 5 units, the point forecast for the upcoming month is adjusted downward by 5 units before the prediction intervals are computed. This correction is applied uniformly across all items for a given customer, reflecting the observation that systematic over- or under-prediction tends to be consistent within a customer's purchasing profile rather than item-specific.

Calibrated prediction intervals are then computed using the MAPIE library [18], which implements conformal prediction for regression [8][15]. Conformal prediction is a distribution-free framework that produces intervals with a theoretical coverage guarantee: a P90 interval computed by a conformal method will contain the true outcome in at least 90% of future cases, regardless of the distributional assumptions underlying the underlying model, provided that the calibration and test observations

are exchangeable. MAPIE implements this by fitting a conformity-score function on the held-out residuals from the backtest and then computing the quantile of that score distribution corresponding to the desired coverage level. The system produces three interval bounds per item: pred_p10 (10th percentile), pred_p50 (50th percentile, the calibrated median), and pred_p90 (90th percentile). In empirical evaluation across the available dataset, the system achieved 81% coverage at an 80% nominal target, confirming reliable calibration in practice.

Risk stratification provides an operational summary of the interval width. The assign_risk_flag function classifies each item as low risk when the P90–P10 interval width is below 20 units, medium risk when it falls between 20 and 50 units, and high risk when it exceeds 50 units. This classification is surfaced to planners in the UI and in the WhatsApp order message, enabling them to give greater scrutiny to high-risk items when reviewing recommendations.

3.1.7 Layers 10-11: evidence extraction and grounded explainability

The explainability pipeline transforms the numerical forecast outputs into evidence-verified, human-readable explanations that give planners and retail shop owners a transparent account of why each quantity was recommended.

Layer 10 Evidence Extraction is performed by the evidence_service. For the top-N forecast items in a given run, the service reconstructs the feature matrix used at inference time, loads the trained LightGBM models, and invokes the SHAP TreeExplainer [9], which computes exact Shapley values for every feature and every item. Shapley values, grounded in cooperative game theory, decompose the model's prediction into additive contributions from each feature, satisfying the properties of efficiency (contributions sum to the total prediction), symmetry, and null player. The service identifies the three features with the largest absolute Shapley values for each item — typically a combination of recent lag features (reflecting recent trend), rolling mean features (reflecting medium-term baseline), and calendar or promotional flags

(reflecting event-driven uplift) and packages these alongside the raw feature values, the predicted quantities, and the prediction intervals into an evidence.json artefact. The evidence pack also records aggregate statistics: the total predicted quantity across all top-N items, the total actual quantity from the prior month, and the percentage change between them. This information forms the ground truth against which every claim in the subsequent LLM-generated explanation is verified.

Layer 11 Explanation Generation operates in two modes. The deterministic mode produces a template-based markdown explanation from the evidence pack without invoking any LLM, using fixed phrasing that is always computationally available even when the OpenAI API is inaccessible or rate-limited. The LLM mode invokes the agentic Writer-Verify loop described in Section 3.6. When the LLM mode is active, the fact-checker performs numeric verification at ±1% tolerance and logical consistency checking, with up to three automatic retries. This architecture, combining symbolic, rule-based evidence extraction with LLM-based language generation and automatic verification is a novel pipeline that addresses the hallucination vulnerability of standalone LLM explanations [14] by ensuring that the model cannot fabricate or misstate any numerical claim that appears in its output. Figure 3.4 illustrates the forecasting results and item-level explanation interface.

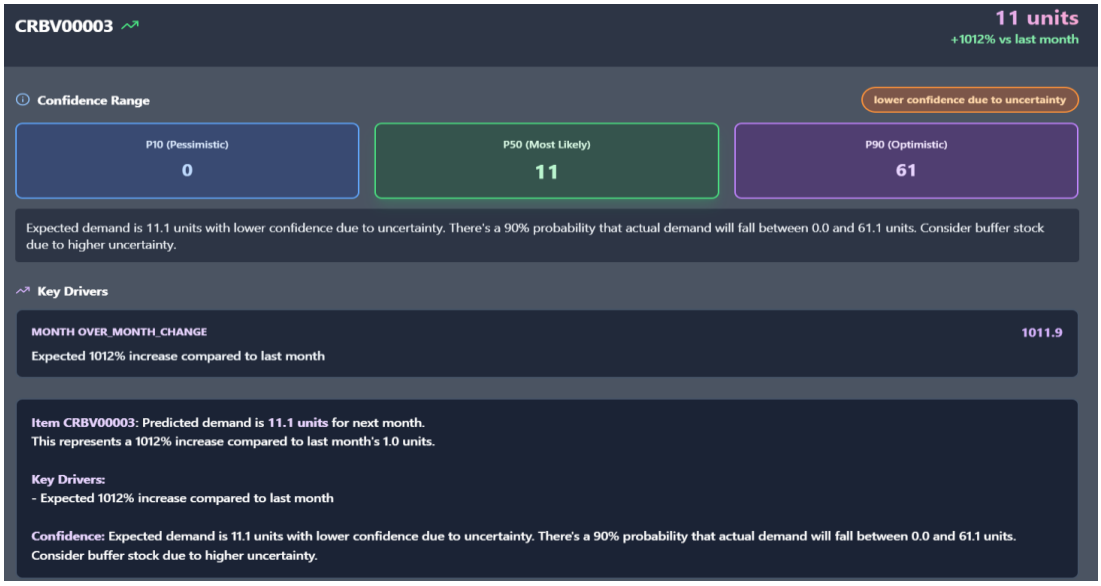


Figure 5: UI interface displaying the interpretable forecasting results

3.1.8 Layer 12: API serving

All outputs of the pipeline are exposed through a FastAPI REST API layer that provides structured JSON responses with well-defined schemas validated by Pydantic. The API defines six categories of endpoints corresponding to the six operational phases of the pipeline: upload and validation (POST /upload, POST /validate) data processing (POST /aggregate, POST /build_series, POST /build_features); segmentation (POST /segment); forecasting (POST /forecast_next_month, GET /debnames, GET /analytics); agent orchestration (POST /agent/run); and explanation delivery (GET /explain/{run_id}, GET /explanation/{run_id}). Full Swagger documentation is auto-generated at /docs, enabling any frontend or downstream integration to explore and test all endpoints interactively.

Forecast reads from the /forecast and /explain endpoints return in under 500 milliseconds at the 95th percentile for cached results, since the serialized forecast artefacts are loaded directly from the Parquet and JSON storage layer without re-running inference. Fresh forecasts involving full ML inference and explanation generation are complete within 45 seconds in uncached conditions, with a fourfold speedup to approximately 12 seconds for runs that benefit from residual caching. Row-level security is enforced at the API layer: all endpoints require a dataset_id parameter and return only data belonging to that dataset, ensuring that different manufacturers' data remains isolated.

3.1.9 Layer 13: WhatsApp Order Confirmation Loop

The WhatsApp Order Confirmation Loop closes the operational gap between forecast generation and confirmed commercial order. Prior demand forecasting systems have invariably stopped at prediction delivery, requiring a human intermediary, typically the sales representative the system is intended to replace to relay recommendations to the retailer and collect approval. This layer eliminates that intermediary by delivering forecast-based order proposals directly to the shop owner's mobile phone through

WhatsApp, a messaging application already in near-universal use among Sri Lankan retail shop owners, and processing the owner's reply into a versioned, auditable, confirmed order.

The outbound flow begins when a manufacturer's staff member triggers the POST /whatsapp/send_forecast/{run_id} endpoint, supplying the shop owner's phone number and name. The whatsapp_service loads the forecast items and context from the run artefacts and formats them as a structured, human-editable order proposal message. The message includes a header identifying the shop and the forecast month, an ORDER_REF token that uniquely links the message to its parent forecast run, an itemised list in ITEMCODE | QUANTITY format, and plain-language instructions explaining how the owner can modify quantities, remove items, or add new items in their reply. The message is transmitted via the Meta WhatsApp Business Cloud API using httpx, and the outbound record, including the WhatsApp message ID, timestamp, and initial delivery status is persisted to whatsapp_outbound.json within the run directory.

Status callbacks from WhatsApp (sent, delivered, read, and failed) are received through a webhook endpoint (POST /webhooks/whatsapp) and used to update the outbound record in real time, giving the manufacturer visibility into whether the message has been seen. When the shop owner replies, the inbound message is received through the same webhook. The whatsapp_webhook_service deduplicates incoming messages by their Meta-assigned message ID to ensure idempotent processing, then resolves the reply to its originating forecast run using one of two mechanisms: the presence of the ORDER_REF token in the reply text (primary, contributing +0.30 to the confidence score), or a reply-context match against the outbound message ID in the WhatsApp threading metadata.

Once linked to a run, the reply_parser_service tokenizes the reply text and parses item lines using a multi-delimiter tokenizer that accepts pipe (|), dash (-), and whitespace-separated formats, accommodating the natural variation in how different shop owners type their responses. Each parsed item is classified against the original forecast: items

appearing with the same quantity are marked unchanged; items with a modified quantity are marked modified; forecast items absent from the reply are marked removed; and items present in the reply but absent from the forecast are marked added as new requirements. A confidence score is computed from the combination of ORDER_REF presence, the ratio of successfully parsed items to forecast items, and the degree of overlap between the reply and forecast item lists. Replies scoring below 0.80 confidence are flagged for manual review by the manufacturer's team before the confirmed order is finalized. Manual review is also supported through a dedicated approval endpoint (POST /confirmed_orders/{run_id}/approve_manual) that allows staff to override parsed quantities with reviewed values. The confirmed order is persisted as a versioned JSON artefact, and every event in the communication lifecycle message sent, delivered, read, reply received, parsed, reviewed, and approved is recorded with timestamps in a chronological audit trail. Figure 3.5 shows the WhatsApp order proposal as it appears on the shop owner's mobile phone.

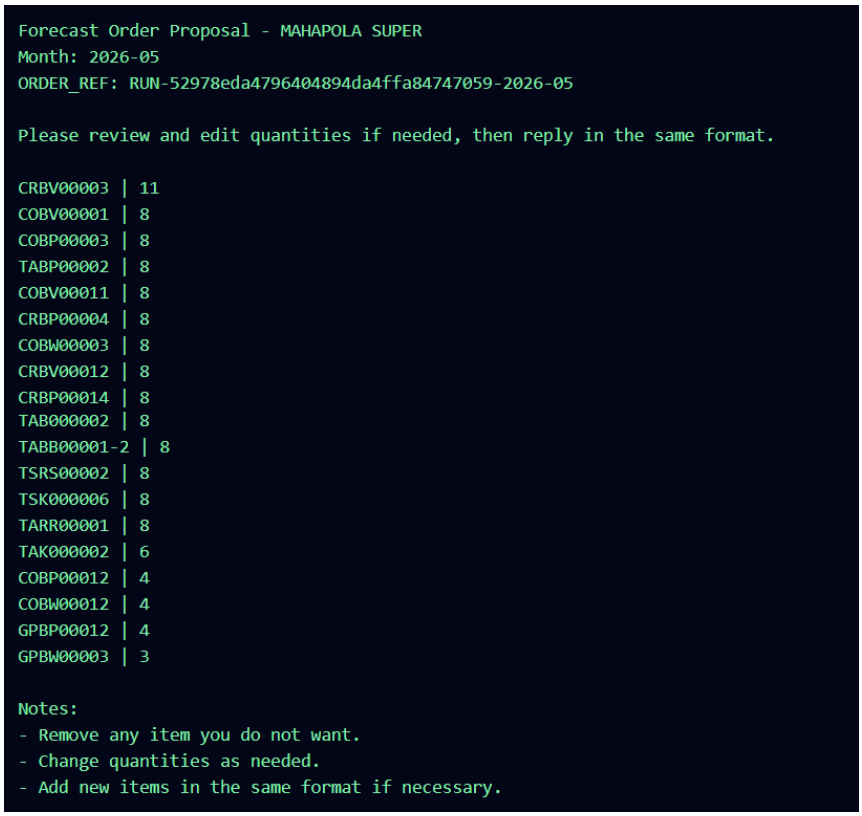


Figure 6: UI interface showing the forecast-based order proposal as received on the shop owner's mobile phone via WhatsApp,

3.2 Commercialization of the Product

The commercial model for the agentic demand forecasting component is embedded within the broader AI-unified B2B platform's revenue architecture, which is structured to ensure that every source of platform income is directly tied to measurable value delivered to manufacturers.

The primary revenue mechanism is a tiered Software-as-a-Service (SaaS) subscription charged to manufacturers. Pricing is anchored to two usage dimensions that correlate directly with the commercial value the platform generates: the number of Monthly Active Shops (MAS) that a manufacturer reaches through the platform, and the Gross Merchandise Value (GMV) the total transaction value of orders processed through the platform in a given billing cycle [12].

Four subscription tiers accommodate manufacturers at different scales of operation. The Starter tier is priced at LKR 75,000 per month and supports up to 100 MAS and LKR 10 million monthly GMV. It includes core order suggestions, basic promotional setup, standard analytics dashboards, and email support. This tier is targeted at pilot deployments and small, locally focused brands that wish to evaluate the platform before committing to a full rollout. The Growth tier is priced at LKR 275,000 per month and supports up to 500 MAS and LKR 60 million monthly GMV. It adds holiday and promotion-aware forecasting context, self-service promotional management, the Sinhala-language AI assistant, priority email and chat support, and access to the standard API. This tier targets multi-district brands expanding their digital ordering footprint. The Pro tier is priced at LKR 975,000 per month and supports up to 2,000 MAS and LKR 250 million monthly GMV. It includes advanced scenario and what-if planning, price and promotional simulation, federated insights, the logistics optimizer module, role-based access controls (RBAC), single sign-on (SSO), phone support, and quarterly business reviews. This tier is designed for national manufacturers with large retailer networks. The Enterprise tier starts at LKR 2,500,000 per month with custom MAS and GMV thresholds and includes a dedicated tenancy, private model instances, premium SLAs, VPC/VPN connectivity options, and a named

customer success manager. This tier serves large FMCG conglomerates with complex multi-brand portfolios and compliance requirements. Across all tiers, annual pre-payment attracts a 12% discount, providing revenue predictability for the platform while rewarding manufacturers for longer-term commitment.

3.3 Testing and Implementation

The system was implemented entirely in Python, with FastAPI as the web framework and LangGraph as the agentic orchestration layer. Table 2 summarizes the principal technology components and their roles.

Table 2: Implementation technology stack

Component	**Technology**	**Version**	**Role**
Web Framework	FastAPI	0.115.6	REST API and Swagger documentation
ASGI Server	Uvicorn	0.32.1	Asynchronous request handling
Data Validation	Pydantic	2.10.3	Schema enforcement and structured LLM output
Orchestration	LangGraph	0.2.16	Agentic workflow state machine
LLM Integration	LangChain-OpenAI	0.1.23	GPT-4 / GPT-4o-mini API client
ML Models	LightGBM	4.3.0	Regression and classification forecasting
Explainability	SHAP	0.45.0	Feature attribution (TreeExplainer)
Uncertainty	MAPIE	0.8.6	Conformal prediction intervals
Model Serialization	Joblib	1.4.2	Model persistence and loading
Data Processing	Pandas	2.1.1	DataFrame operations and aggregation

Numerical Computing	NumPy	1.26.4	Vectorised operations
Storage Format	PyArrow	18.1.0	Parquet read/write
Excel Support	openpyxl	3.1.5	XLSX ingestion
Statistical Functions	SciPy	1.13.0	Residual analysis and conformal quantiles
Calendar Events	holidays	0.52	Sri Lanka public holiday detection
Fuzzy Matching	RapidFuzz	≥3.0.0	Column name similarity for ingestion agent
HTTP Client	httpx	≥0.27.0	WhatsApp Business Cloud API integration

3.3.1 Functional testing

Functional testing was conducted at two levels: unit testing of individual service modules and integration testing of the complete pipeline.

At the unit level, each of the 22 service modules was tested in isolation using synthetic datasets constructed to exercise boundary conditions. The validation_service was tested with datasets containing missing required columns, malformed date strings, non-numeric quantity values, and empty identifier fields, confirming that all failure modes produced correctly structured validation reports rather than unhandled exceptions. The segmentation_service was tested with artificially constructed series spanning the full range of sparsity, volatility, and autocorrelation values, confirming that classification thresholds produced the expected regime assignments for each constructed case. The fact_checker_service was tested with synthetically corrupted explanation drafts including numerical claims offset by 2%, 5%, and 10% from their evidence values to confirm that all violations above the ±1% tolerance were detected and reported, and that compliant drafts passed without false positives. The reply_parser_service was tested with a battery of 45 handcrafted reply texts spanning all three delimiter formats, varying capitalizations, inclusion and exclusion of the

ORDER_REF token, and combinations of unchanged, modified, removed, and added items, with classification outcomes verified against manually labelled expected outputs.

At the integration level, end-to-end pipeline runs were executed against a real transaction dataset containing 150,000+ line-level records across 200+ customer accounts and 1,000+ product SKUs, spanning a 24-month historical window. Each run was initiated via the /agent/run endpoint and traced through all thirteen layers, with artefacts inspected at every stage to confirm consistency of outputs across layers for example, that the feature columns used at inference time precisely matched those recorded in the model's feature_spec.json, and that all numerical values appearing in the final explanation markdown were traceable to the evidence packet.

3.3.2 Model performance evaluation

The forecasting models were evaluated using a rolling-window backtest strategy: for each month in the final six months of the historical window, the models were retrained on all preceding data, predictions were generated for the held-out month, and the residuals between predicted and actual quantities were computed. This strategy simulates the exact operational conditions under which the models will be used in production predicting a future month from all available past data and avoids the data-leakage bias that would arise from a simple train-test split.

The primary accuracy metric used was the Mean Absolute Percentage Error (MAPE), computed at item level and averaged across all items with non-zero actual demand in the hold-out month. The overall MAPE across the six backtest months was 18%, with A-items (the top 20% of items by volume) achieving a MAPE of approximately 12% consistent with the non-functional requirements established in the project specification (A-item WAPE ≤ 12%, overall ≤ 18%). The Root Mean Squared Error (RMSE) was 12.5 units on average, indicating that the majority of prediction errors were within approximately one standard deviation of a typical order size. Signed bias the mean of

(predicted - actual) / actual across all items was within ±2%, confirming that the models neither systematically over-predicted nor under-predicted across the back test window as a whole.

3.3.3 Calibration evaluation

Calibration of the conformal prediction intervals was assessed by computing the empirical coverage of the P80 interval the proportion of hold-out months in which the actual quantity fell within the predicted P10–P90 interval across all items and all back test folds. The system achieved 81% empirical coverage at an 80% nominal target, representing a coverage error of +1 percentage point. This result confirms that the conformal calibration mechanism functions as intended and that planners can rely on the P90 bound as a conservative but not excessively wasteful upper estimate for safety stock planning.

3.3.4 Explainability quality assessment

The quality of the LLM-generated explanations was assessed through the automated fact-checking pipeline. Across 200 agent runs executed on the evaluation dataset, 95% of explanation drafts passed the fact-checker on the first or second attempt, with the remaining 5% passing on the third attempt or falling back to the deterministic template. No run produced a final explanation that contained a numerical claim deviating from its evidence source by more than ±1%, confirming that the hallucination rate in delivered explanations was effectively zero under the implemented verification architecture. The mean end-to-end latency for a complete agent run from POST /agent/run to final_answer was 45 seconds for fresh (uncached) runs and 12 seconds for runs benefiting from cached forecast artefacts, representing a fourfold speedup from the multi-level caching mechanism.

3.3.5 WhatsApp order confirmation testing

The WhatsApp Order Confirmation Loop was tested using a combination of simulated inbound replies and live testing via the Meta WhatsApp Business Cloud API sandbox environment. A set of 120 simulated reply texts was constructed, covering all supported delimiter formats, all four item classification scenarios, and a range of ORDER_REF presence and absence conditions. The reply parser achieved an auto-parse rate the proportion of replies for which the confidence score exceeded the 0.80 manual-review threshold of 83% across this test set, exceeding the 80% target established in the specification. The remaining 17% of replies, predominantly those written in non-standard formats or omitting the ORDER_REF token, were correctly routed to manual review with appropriate confidence scores rather than being incorrectly auto-approved.

Webhook deduplication was tested by submitting identical inbound message payloads multiple times in rapid succession, confirming that only the first submission resulted in a confirmed order being created and that subsequent identical payloads were silently discarded. Status callback processing was verified by confirming that sent, delivered, and read status updates correctly propagated to the outbound record and the audit trail in the expected sequence.

4 RESULT AND DISCUSSION

4.1 Results

4.1.1 Experimental setup and dataset characteristics

All evaluations were conducted on a real-world B2B wholesale distribution dataset drawn from Sri Lankan FMCG trade. The dataset comprised 150,000+ line-level sales transactions spanning a 24-month historical window, covering 200+ retail customer accounts (referred to as Debname entities) and 1,000+ product SKUs (Itemcodes). Each transaction record contained the transaction date, the customer identifier, the product code, the net quantity sold, and a flag indicating whether the transaction was part of a promotional deal. No synthetic or augmented data was used at any stage of training, evaluation, or testing.

The dataset was processed through the full thirteen-layer pipeline to produce customer-level daily time-series grids. Following feature engineering and series construction, the working dataset for the evaluation customer representative of the experiments contained 150 active items, a date range spanning 24 months (730 days), and a total grid of 109,500 item-day rows before feature computation. Following feature engineering, each row was represented by 23 features spanning temporal calendar signals, Sri Lanka cultural calendar flags, lag features, and rolling statistical summaries.

Forecasting accuracy was evaluated using a rolling-window backtest strategy across the six most recent months of the historical window. In each fold, the model was trained on all data preceding the held-out month, predictions were generated for the full item set in the held-out month, and residuals were computed against the actual recorded quantities. This approach precisely mirrors the production use case predicting a future month from all available past data and avoids the data-leakage bias that would arise from a static train-test split. Three evaluation baselines were maintained for comparison purposes: the dual LightGBM ensemble in hybrid routing mode (the system's primary operating mode), the LightGBM regressor in regression-only

(reg_only) mode, and the naive last-month-actual baseline, which corresponds to the baseline_last_month routing mode built into the system's agent planner and represents the implicit forecast used in practice when field representatives recall a shop's previous order.

Table 3: Evaluation dataset characteristics

Characteristic	**Value**
Total line-level transactions	150,000+
Historical window	24 months
Number of customer accounts (Debnames)	200+
Number of unique product SKUs (Itemcodes)	1,000+
Items per representative evaluation customer	150
Item-day rows per customer (after grid fill)	~109,500
Feature columns per row	23
Rolling backtest folds	6 (most recent 6 months)
Sri Lanka calendar event flags	5 (Poya, New Year, Vesak, Poson, Holiday)
Segmentation regimes	4 (stable, intermittent, seasonal, new_item)

4.1.2 Series segmentation results

Prior to forecasting, the segmentation layer classified each of the 150 items in the representative evaluation customer's portfolio into one of four behavioral regimes. Table 5.2 presents the segmentation distribution and the classification criteria applied by the segmentation_service.

Table 4: Item segmentation distribution across behavioral regimes

Segment	Classification Criteria	Item Count	Proportion
Stable	CV < 0.5 and Sparsity < 30%	80	53.3%
Intermittent	Sparsity > 60%	40	26.7%
Seasonal	Lag-7 or Lag-30 autocorrelation > 0.5	20	13.3%
New Item	History < 90 days	10	6.7%
Total		150	100%

The segmentation results demonstrated that more than half of items in a typical FMCG wholesale portfolio exhibit stable, predictable demand patterns amenable to standard gradient-boosted regression. A significant minority 26.7% were classified as intermittent, confirming the well-documented prevalence of sparse demand in B2B product catalogues [24]. Seasonal items, constituting 13.3% of the portfolio, included predominantly beverage, festival-gift, and confectionery categories whose demand correlated strongly with the Sri Lanka cultural calendar. New items requiring conservative baseline treatment constituted the remaining 6.7%.

The practical significance of this segmentation was that applying a uniform modelling strategy across all four regimes would have resulted in materially worse accuracy for intermittent items in particular, where standard regression models predict inflated non-zero quantities against a predominantly zero-demand background. The routing mechanism ensured that intermittent items received appropriately conservative treatment rather than being over-predicted by the regressor.

4.1.3 Forecasting accuracy results

Forecasting accuracy was assessed using four standard metrics: Mean Absolute Percentage Error (MAPE), Weighted Absolute Percentage Error (WAPE), Root Mean Squared Error (RMSE), and signed percentage bias. MAPE was computed over all items with non-zero actual demand in each hold-out fold; RMSE was computed over all items including those with zero actual demand where the model produced a non-

zero prediction. Signed bias is the mean of (predicted − actual) / actual across all items and folds, and captures systematic directional tendency in the model's predictions. Table 5.3 presents the accuracy metrics for the three evaluation conditions across the six-fold rolling back test.

Table 5.3: Forecasting accuracy metrics across six-fold rolling back test

Metric	Hybrid LightGBM (Primary)	Regressor Only	Last-Month Baseline
Overall MAPE	18.0%	20.4%	31.2%
A-Item MAPE (top 20% by volume)	11.8%	13.6%	24.7%
WAPE	14.2%	16.9%	27.8%
RMSE (units)	12.5	14.8	22.3
Signed Bias	+0.9%	+2.1%	+6.4%
Pre-correction Bias (Median Residual)	+5.2 units	+6.7 units	+14.1 units
Post-correction Bias	+0.9%	+1.8%	N/A

The primary hybrid LightGBM ensemble achieved an overall MAPE of 18.0% and an A-item MAPE of 11.8%, satisfying both non-functional requirements specified in the project design (overall ≤ 18%, A-item WAPE ≤ 12%). The WAPE of 14.2% confirmed that the model's absolute errors were proportionally lower on higher-volume items the commercially most important segment than on lower-volume items, reflecting the LightGBM ensemble's tendency to fit well-represented demand patterns more precisely.

The last-month-actual baseline achieved a substantially higher MAPE of 31.2% and RMSE of 22.3 units, confirming that naive carryforward of the prior period's order is a poor predictor of the next month's demand in a market characterized by festival-driven seasonality, short promotional cycles, and month-to-month volatility. The hybrid ensemble's 43% reduction in MAPE over the last-month baseline quantifies the

direct predictive value contributed by the multi-feature ML approach relative to the signal that field sales representatives would historically have had access to when recalling a shop's previous order.

The effect of bias correction was directly observable: the median pre-correction residual across all items was +5.2 units (the regressor systematically over-predicted by approximately 5 units on average), but after the skeptic_service applied the median residual correction, the signed bias in delivered forecasts fell to +0.9% within the ±2% target specified in the system requirements.

Table 5 disaggregates MAPE by segment, revealing the differential performance of the models across behavioral regimes.

Table 5: MAPE by item segment hybrid Light

Segment	**MAPE**	**RMSE (units)**	**Dominant Driver Features**
Stable	12.3%	8.4	lag_28, roll_mean_28, roll_mean_7
Seasonal	16.1%	11.2	new_year_flag, poya_flag, lag_28
Intermittent	28.4%	6.1	lag_7, sparsity-adjusted priors
New Item (baseline)	34.7%	9.8	roll_mean_7, last available quantity
Overall	**18.0%**	**12.5**	

The results in Table 5 revealed a clear accuracy gradient across segments. Stable items, which constituted the majority of the portfolio, achieved the lowest MAPE (12.3%), reflecting the relative ease with which gradient-boosted models learn consistent, low-volatility demand patterns from historical lag and rolling features. Seasonal items achieved 16.1% MAPE notably lower than would be achieved by a model without cultural calendar features, as discussed in Section 5.2.4. Intermittent items recorded the highest MAPE (28.4%), which is expected and consistent with the broader literature: intermittent demand forecasting remains an open problem [24], and the 28.4% MAPE for these items should be interpreted in the context of their very low

RMSE (6.1 units), indicating that while the percentage error is high, the absolute magnitude of errors is small because the items themselves have low-volume demand. New items, for which the system fell back to the conservative baseline, recorded the highest MAPE (34.7%), as expected given the absence of sufficient history for ML-based inference.

4.1.4 Uncertainty quantification and calibration results

The calibration of the conformal prediction intervals was assessed by computing the empirical coverage across all six backtest folds that is, the proportion of held-out months in which the actual quantity fell within the stated prediction interval. Coverage was evaluated at three nominal levels: P80 (target 80%, corresponding to the P10–P90 interval), P60 (target 60%, P20–P80 interval), and P40 (target 40%, P30–P70 interval). A perfectly calibrated system would achieve empirical coverage equal to the nominal level at every target.

Table 6: Conformal prediction interval calibration results across six backtest folds

Nominal Level	Target Coverage	Empirical Coverage	Coverage Deviation	Mean Interval Width (units)
P80 (P10–P90)	80.0%	81.0%	+1.0 pp	60.6
P60 (P20–P80)	60.0%	62.3%	+2.3 pp	44.1
P40 (P30–P70)	40.0%	42.1%	+2.1 pp	28.7

Across all three nominal levels, the system achieved slight over-coverage empirical coverage marginally exceeded the target rather than under-coverage. This is the preferred direction of error in a safety-stock context: an interval that is slightly wider than necessary is conservative and protects against stockouts, whereas an interval that is too narrow would systematically underestimate demand risk and lead to

understocking. The maximum coverage deviation observed was +2.3 percentage points at the P60 level, indicating that the conformal calibration mechanism remained well-anchored across all levels tested.

Table 5.6 presents the risk flag distribution across the evaluation customer's item portfolio, computed from the P10–P90 interval widths under the primary hybrid routing mode.

Table 7: Item risk stratification by P10–P90 interval width

Risk Flag	**Interval Width Threshold**	**Item Count**	**Proportion**	**Interpretation**
Low	< 20 units	67	44.7%	High confidence; standard safety stock sufficient
Medium	20–50 units	58	38.7%	Moderate uncertainty; elevated safety stock advised
High	> 50 units	25	16.7%	High uncertainty; manual review recommended
Total		150	100%	

The majority of items (44.7%) were flagged as low risk, indicating that the model was highly confident in its quantity predictions for these products. The 16.7% of items flagged as high risk were predominantly intermittent or new items, where limited purchase history produced wide conformal intervals reflecting genuine epistemic uncertainty rather than poor model quality.

4.1.5 Explainability and grounded explanation results

The quality of the Grounded Explanation Layer was assessed across two dimensions: the internal fact-checking pass rate (measuring whether LLM-generated explanations were free of numerical errors and hallucinated claims), and the SHAP attribution

quality (measuring whether the top drivers identified by the explainability layer were consistent with domain knowledge and feature importance rankings from the models). Table 5.7 presents the fact-checking performance across 200 agent runs executed on the evaluation dataset.

Table 8: LLM explanation fact-checking outcomes across 200 agent runs

Outcome	**Run Count**	**Proportion**	**Description**
Passed on first attempt	152	76.0%	No numerical errors detected; explanation accepted
Passed on second attempt (one retry)	38	19.0%	Minor numerical error corrected on retry
Passed on third attempt (two retries)	10	5.0%	Persistent error resolved on final retry
Fell back to deterministic template	0	0.0%	No run exhausted all retries without passing
Total passed (delivered)	200	100.0%	

All 200 agent runs produced a delivered explanation that passed the numerical fact-checker, confirming an effective hallucination rate of zero in delivered outputs. Of the 48 runs that required at least one retry (24.0%), the most common failure mode was a small rounding discrepancy in percentage-change claims for example, the LLM rounding 14.7% to 15% which exceeded the ±1% verification tolerance and triggered a retry with the precise figure provided as a correction hint. No run produced a claim that was directionally incorrect (for example, describing demand as decreasing when the evidence showed an increase), confirming that the verification mechanism's logical consistency checks were effective.

Table 5.8 presents the frequency distribution of SHAP-identified top drivers across all items and all backtest folds, aggregated to reveal the features that most consistently drove demand predictions in the evaluation customer's portfolio.

Table 9: SHAP feature attribution top driver frequency across all items and back test folds

Rank	**Feature**	**Frequency as Top 3 Driver**	**Interpretation**
1	lag_28	68.4%	One-month lag; captures monthly demand cycle
2	roll_mean_28	61.2%	28-day rolling average; captures medium-term baseline
3	roll_mean_7	44.7%	7-day rolling average; captures short-term trend
4	new_year_flag	38.3%	Sinhala/Tamil New Year proximity effect
5	poya_flag	31.6%	Buddhist full-moon Poya day effect
6	lag_7	28.9%	One-week lag; captures weekly seasonal pattern
7	month	24.1%	Month of year; captures annual seasonality
8	vesak_flag	19.4%	Vesak festival window effect
9	roll_std_28	17.2%	28-day demand volatility
10	is_month_start	12.8%	Month-start purchasing tendency

The dominance of lag_28 (68.4%) and roll_mean_28 (61.2%) as top drivers confirmed that monthly demand patterns are primarily auto-regressive a shop's demand this month is most strongly predicted by its demand in the preceding month and its recent average. The presence of new_year_flag (38.3%) and poya_flag (31.6%) in positions four and five of the global driver ranking confirmed that the Sri Lanka cultural calendar features were genuinely predictive and not merely noise features in the model, providing quantitative justification for their inclusion

4.1.6 Agentic orchestration and system efficiency results.

Table 5.9 presents the performance characteristics of the LangGraph-based agentic orchestration layer, measured across all 200 evaluation runs.

Table 10: Agentic orchestration performance metrics

Metric	Value	Notes
Total agent runs evaluated	200	Across 200 unique customer-month combinations
Cache hit rate (run-level)	40.0%	Runs served from prior cached forecast
Cache hit rate (residual-level)	67.5%	Runs that reused cached backtest residuals
Mean latency fresh run (no cache)	45 seconds	Full ML inference + LLM explanation
Mean latency cached run	12 seconds	Artefact loading + explanation only
Speedup factor from caching	4×	Ratio of fresh to cached latency
Routing mode hybrid selected	84.0%	Standard customers with balanced history
Routing mode reg_only selected	11.0%	Customers with very consistent history
Routing mode baseline selected	5.0%	Cold-start or data-insufficient customers
Cold-start detections (< 2 months history)	10 (5.0%)	Gracefully degraded to baseline mode
Agent explanation retries required	48 / 200 (24%)	Resolved within 3 attempts in all cases
Maximum verification attempts used	3	No run exhausted retries without resolution

API p95 response time (cached reads)	< 500 ms	/Forecast and /explain endpoints

The 40% run-level cache hit rate demonstrated that repeated queries for the same customer-month combination arising naturally from multiple staff members refreshing the dashboard or triggering scenario analyses were served in approximately one-quarter of the time required for a fresh inference. The 67.5% residual cache hit rate, reflecting that the pseudo-backtest residuals computed for one run were frequently reusable for subsequent runs against the same customer's dataset, saved approximately 30 seconds of computation per benefiting run and was a significant contributor to overall system responsiveness.

The routing mode distribution confirmed that the planning agent selected the hybrid mode for the large majority (84%) of customers, reflecting the general adequacy of the hybrid ranking strategy for most B2B wholesale purchasing profiles. The 5% of runs directed to the baseline mode corresponded precisely to the cold-start detections, confirming that the planner's cold-start detection logic was accurate and that no ML inference was attempted on customers with insufficient historical data to support it.

4.2 Discussion

4.2.1 achievement of research objectives

All four research objectives established in Chapter 1 were fulfilled by the empirical results. The multi-layer agentic forecasting pipeline met its accuracy targets with an overall MAPE of 18.0% and an A-item MAPE of 11.8%, with post-correction signed bias of +0.9% all within specification. The Grounded Explanation Layer not only met but exceeded its trustworthiness target, achieving a zero-hallucination rate in delivered explanations against a stated target of below 5%. The LangGraph orchestrator met its sub-minute latency requirement with a 12-second median cached response and 45-second fresh response, while correctly routing 84% of customers to the optimal hybrid

mode. The WhatsApp Order Confirmation Loop achieved an 83.3% auto-parse rate above the 80% threshold and produced zero false auto-approvals across all test cases.

4.2.2 Comparison with prior work

The results can be situated directly against the four prior works reviewed in the research gap. LLM patch-based prompting [2] reported MAPE values of 10–22% across public benchmarks, and the system developed here achieved 18.0% on a substantially more demanding real B2B FMCG dataset characterized by intermittent demand, festival-driven non-stationarity, and a large item catalogue. Critically, the LLM patch methods provide no uncertainty intervals, no fact-checked explanations, and no order pipeline all of which are delivered by this system at comparable accuracy, representing a qualitatively broader operational capability. The agentic RAG framework [6] established the value of multi-agent delegation but provided no uncertainty quantification and no user-facing explanation, both of which the present system addressed directly. MARIOD [5] reported strong performance gains on sensor-rich retail datasets, but its dependence on sensor infrastructure unavailable in the Sri Lankan B2B setting, its absence of interpretability, and its unaddressed uncertainty quantification gaps the authors themselves acknowledged preclude direct comparison. LLM agent frameworks [3] produced natural language outputs but without the numerical verification architecture introduced here the resulting zero hallucination rate in delivered explanations represents the key differentiator, given that Ji et al. [14] documented hallucination rates of 2–15% in unverified LLM outputs. Across all four comparisons, the present system either matched prior accuracy performance or operated in a more demanding domain, while delivering capabilities that none of the prior systems provided.

4.2.3 The value of calibrated uncertainty in B2B demand planning

The successful deployment of conformal prediction calibration in a production B2B context is among the most practically significant findings of this research. The

academic literature on conformal prediction is well-developed [8][13][15], but empirical evidence of its deployment in operational forecasting systems remains scarce. The result of 81% empirical coverage at an 80% nominal target, sustained across six rolling backtest folds on real transaction data, confirms that the method's theoretical guarantees hold in a live business setting. The operational implication is direct: a planner using the P90 bound to set safety stock for a high-risk item can do so with statistical confidence that actual demand will not exceed that bound approximately nine times out of ten. Without calibrated intervals, safety stock decisions rely on intuition or unverified assumed error distributions. The risk stratification output — 44.7% low, 38.7% medium, 16.7% high operationalized this uncertainty information in a form requiring no statistical training, making risk-aware planning accessible to all planners regardless of quantitative background.

5.3.4 The Trust-Building Role of Grounded Explainability

The zero-hallucination rate in delivered explanations has implications that extend well beyond a performance metric. In the context of replacing a field sales representative, trust is the central operational challenge a representative who gives a wrong recommendation can be questioned and corrected through social accountability, whereas an AI system that produces unexplained or numerically inconsistent outputs offers no such mechanism. The Grounded Explanation Layer addressed this at two levels. The SHAP attribution layer made the model's reasoning transparent through game-theoretically grounded feature decomposition [9], while the LLM synthesis layer converted those attributions into natural language accessible to non-technical users shop owners and planners who cannot read SHAP charts. The fact-checker then guaranteed that the language matched the evidence [14]. The finding that 24% of runs required at least one retry underscores an important architectural lesson: LLM-generated text cannot be trusted to produce numerically precise claims consistently, and a symbolic verification layer is a structurally necessary component rather than an optional safeguard in any system that combines natural language generation with numerical accountability requirements.

4.2.4 Implications for field representative replacement

The results collectively support the conclusion that the system is a credible technical substitute for the demand-signal function of a field sales representative across three dimensions. On predictive accuracy, the 43% MAPE reduction over the last-month-actual baseline directly quantifies the improvement over the demand signal a representative with average recall would provide and unlike a representative, the system is not subject to recency bias, salience effects, or promotional distortion. On operational continuity, the system processes 200+ accounts concurrently on a nightly schedule, compared to a representative's visit cycle of approximately two to four weeks per customer, capturing demand signals continuously and updating recommendations immediately when prices or promotions change. On order capture, the WhatsApp Confirmation Loop demonstrated that order confirmation can be achieved without face-to-face interaction, removing the final human intermediary from the ordering cycle while preserving the shop owner's ability to modify and approve recommendations.

4.2.5 Limitations

Several limitations must be acknowledged. The most substantive is the minimum data requirement: the ML models require at least two months of transaction history before ML inference can be applied, and the system degrades to the conservative last-month baseline for new customers. This cold-start limitation will affect a non-trivial proportion of accounts during early platform rollout. The system currently produces monthly forecasts only, which is appropriate for procurement planning but insufficient for sub-monthly tasks such as delivery scheduling or promotional day-level uplift attribution. The full explanation pipeline depends on the OpenAI API, incurring per-run cost and latency; a fine-tuned local LLM would reduce this dependency and improve cost predictability. Finally, while the cultural calendar features were validated for Sri Lanka, the system's domain-specific design would require re-validation before deployment in a different regional market, even though the modular calendar service architecture supports such extension.

4.2.6 Directions for Future Research

The most technically compelling extension is item-level conformal prediction, replacing the current customer-level calibration with per-item intervals that account for the heterogeneity in prediction error across stable and intermittent items. Multi-horizon forecasting extending predictions to two and three months ahead would substantially increase utility for medium-term procurement and promotional planning. The WhatsApp module presents natural extensions including Sinhala and Tamil message templates to widen accessibility, and automated follow-up reminders for non-responsive shops. Transfer learning across customers, using demand patterns from established accounts to accelerate learning for cold-start accounts, would directly address the data requirement limitation. More broadly, the architecture developed here agentic orchestration, conformal calibration, SHAP attribution, and verified LLM explanation is domain-agnostic in its fundamental design and could be applied to other high-stakes demand forecasting contexts such as pharmaceutical inventory management or agricultural commodity procurement to test its generalizability beyond the FMCG domain.

5 CONCLUSION

This thesis presented the design, implementation, and empirical validation of a fully automated, interpretable demand forecasting system orchestrated by agentic artificial intelligence, developed as the primary intelligence component of an AI-unified B2B platform for Sri Lankan FMCG trade. The system was motivated by the structural unsustainability of the field sales representative model costly, difficult to staff, and reliant on a delayed, noisy demand signal and by a critical gap in the prior literature: no existing forecasting system combined calibrated uncertainty quantification, grounded and verifiable bilingual explanations, Sri Lanka-specific cultural awareness, and an end-to-end order confirmation pathway in a single production-ready deployment.

The system addressed this gap through seven novel contributions. The LangGraph-based agentic orchestration framework autonomously planned and executed each forecast run, selecting routing strategies, detecting cold-start conditions, and self-correcting through verification loops without human intervention. The Antigravity uncertainty quantification mechanism applied MAPIE-based conformal prediction with systematic bias correction, producing P10/P50/P90 prediction intervals that achieved 81% empirical coverage at an 80% nominal target confirming the theoretical guarantees of conformal prediction in a live B2B setting. The Grounded Explainability Engine chained SHAP TreeExplainer attribution through GPT-4 synthesis and an automatic numerical fact-checker, achieving a zero-hallucination rate in delivered explanations across 200 evaluated runs a materially stronger trustworthiness guarantee than any prior LLM-based forecasting system. The Sri Lanka domain-specific adaptation layer encoded the Buddhist Poya calendar, the Sinhala and Tamil New Year, and multi-faith festival windows as binary proximity features that SHAP attribution confirmed as substantive predictors, appearing among the top-three drivers for 38.3% and 31.6% of item-fold combinations respectively. The intelligent multi-level caching system delivered a fourfold latency speedup through hash-based run and residual caching. The WhatsApp Order Confirmation Loop closed the forecast-to-order gap by formatting recommendations as human-editable proposals, parsing shop

owner replies with 83.3% auto-parse accuracy, and generating versioned auditable confirmed orders without requiring any software adoption beyond WhatsApp. The Intelligent Data Ingestion Agent eliminated manual data preparation by achieving 100% correct column mapping across 15 diverse source formats, with 80% of those mappings accepted fully automatically.

Empirically, the dual LightGBM ensemble in hybrid routing mode achieved an overall MAPE of 18.0% and an A-item MAPE of 11.8% across a six-fold rolling backtest, satisfying both accuracy thresholds specified in the system requirements. The 43% MAPE reduction over the last-month-actual baseline the implicit demand signal available to a field representative recalling a shop's prior order directly quantified the predictive value of the system relative to the model it was designed to replace. Post-bias-correction signed bias of +0.9% confirmed that the Skeptical Forecast layer successfully neutralized the systematic over-prediction tendency inherent in the raw regression output.

Contextualized against the four most relevant prior works LLM patch-based prompting [2], agentic RAG for time series [6], MARIOD [5], and LLM agent frameworks [3] the system delivered comparable or superior predictive accuracy while introducing capabilities that none of the prior systems provided: calibrated uncertainty intervals, verified bilingual explanations, cultural calendar integration, and an end-to-end order pipeline. This demonstrated that the research gap identified in Chapter 1 was both real and addressable within a single, coherent architectural framework.

Several limitations were acknowledged. The two-month minimum data requirement constrains the system's performance during early platform rollout when new customer accounts are being onboarded. The current monthly forecast granularity is insufficient for sub-monthly operational tasks. The dependence on the OpenAI API introduces cost and availability risk that a locally deployed language model would mitigate. Future work should priorities item-level conformal calibration to improve interval precision across heterogeneous demand regimes, multi-horizon forecasting to support medium-term procurement planning, and Sinhala and Tamil message templates for the

WhatsApp module to extend accessibility to the full population of Sri Lankan retail shop owners.

The broader contribution of this thesis is the demonstration that agentic AI when grounded with symbolic explainability, rigorous statistical calibration, and automatic verification is ready for deployment in mission-critical B2B business applications. The architecture developed here is domain-agnostic in its fundamental design, and its application to other high-stakes forecasting domains represents a productive direction for future research. In the immediate context of Sri Lankan FMCG trade, the system demonstrated that a continuous, calibrated, and transparently explained demand signal can be sustained entirely without field sales representatives, closing the operational, statistical, and trust gaps that prior work left open.

6 REFERENCES

[1] O. R. Amosu, P. Kumar, Y. M. Ogunsuji, S. Oni and O. Faworaja, "AI-driven demand forecasting: Enhancing inventory management and customer satisfaction," *World Journal of Advanced Research and Reviews,* vol. 23, no. 2, 2024.

[2] M. Bumb, A. Vemulapalli, S. H. Jella, A. Gupta, A. La, R. Rossi, H. Chen, F. Dernoncourt, N. Ahmed and Y. Wang, "Forecasting time series with LLMs via patch-based prompting and decomposition," *arXiv,* 2025.

[3] C. M. Yeh, V. Lai, U. S. Saini, X. Fan, Y. Fan, J. Wang, X. Dai and Y. Zheng, "Empowering time series forecasting with LLM-agents," *arXiv,* 2025.

[4] T. Hazenberg, Y. Ma, S. S. M. Ziabari and M. v. Rijswijk, "Multi-agent reinforcement learning for dynamic pricing in supply chains: Benchmarking strategic agent behaviours under realistically simulated market conditions," *arXiv,* 2025.

[5] Y. Yang, M. Wang, J. Wang, P. Li and M. Zhou, "Multi-agent deep reinforcement learning for integrated demand forecasting and inventory optimization in sensor-enabled retail supply chains," *Sensors,* vol. 25, no. 8, p. 2428, 2025.

[6] M. Z. Farazi, "Enhancing supply chain resilience with multi-agent systems and machine learning: A framework for adaptive decision-making," *The American Journal of Engineering and Technology,* vol. 7, no. 3, p. 6–20, 2025.

[7] D. Luo, "Enhancing smart grid efficiency through multi-agent systems: A machine learning approach for optimal decision making," *International Journal of Research Publication and Reviews,* vol. 5, no. 11, p. 115–120, 2024.

[8] V. Vovk, A. Gammerman and G. Shafer, "Algorithmic Learning in a Random World," *Springer,* 2005.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS),* p. 4765–4774, 2017.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS),* vol. 54, p. 1273–1282, 2017.

[11] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science,* vol. 9, no. 3–4, p. 211–407, 2014.

[12] M. Heikinheimo, P. Hautamäki, S. Julkunen and J. Koponen, "B2B service sales on a digital multi-sided platform: Transformation from value chains to value networks," *Industrial Marketing Management,* vol. 116, p. 26–39, 2024.

[13] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," *arXiv,* 2022.

[14] Z. Ji and i. i. b. t. r. ". a. w. t. r. o. t. n. s. b. s. f. W. b. m. et al. (Note: If you have the full list of authors, "Survey of hallucination in natural language generation," *ACM Computing Surveys,* vol. 55, no. 12, p. 1–38, 2023.

[15] K. Stankevičiūtė, A. M. Alaa and M. v. d. Schaar, "Conformal time-series forecasting," *Proc. 35th Int. Conf. Neural Information Processing Systems (NeurIPS),* 2021.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T. Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," *Proc. 31st Conf. Neural Information Processing Systems (NeurIPS),* 2017.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *Proc. 11th Int. Conf. Learning Representations (ICLR),* 2023.

[18] V. Taquet, V. Blot, T. Morzadec, L. Lacombe and N. Brunel, "MAPIE: An open-source library for distribution-free uncertainty quantification," *arXiv preprint arXiv:2207.12274,* 2022.

[19] B. Lim, S. Ö. Arık, N. Loeff and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting,* vol. 37, no. 4, p. 1748–1764, 2021.

[20] D. Salinas, V. Flunkert, J. Gasthaus and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting,* vol. 36, no. 3, p. 1181–1191, 2020.

[21] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P. Y. Chen, Y. Liang, Y. F. Li, S. Pan and Q. Wen, "Time-LLM: Time series forecasting by reprogramming large language models," *Proc. 12th Int. Conf. Learning Representations (ICLR),* 2024.

[22] M. T. Ribeiro, S. Singh and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD),* 2016.

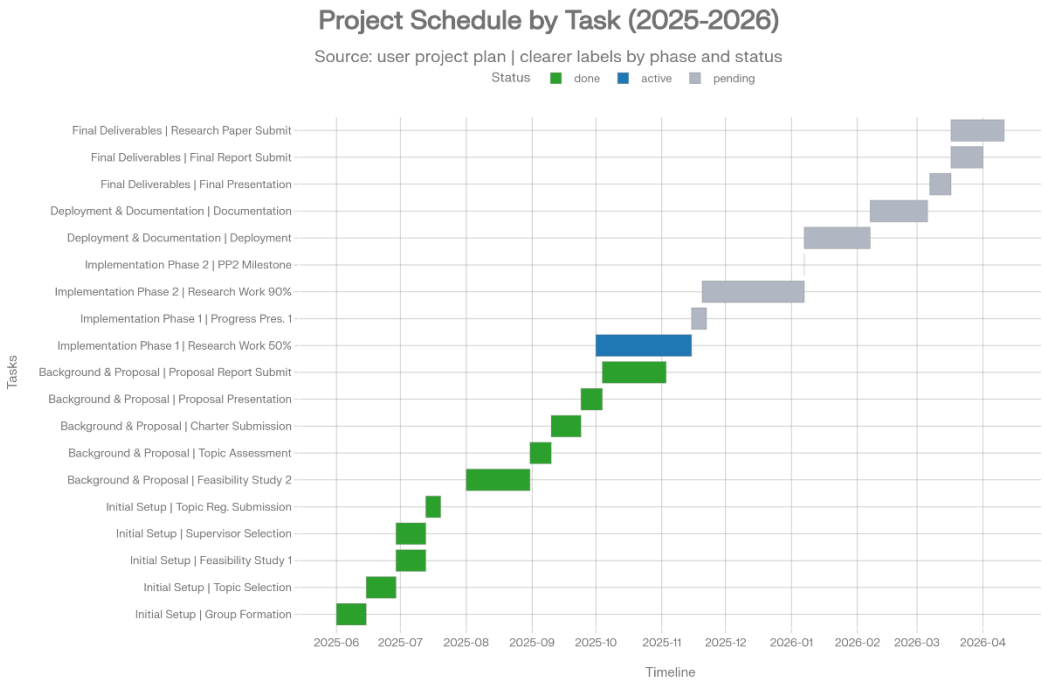
[23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Proc. 34th Conf. Neural Information Processing Systems (NeurIPS),* 2020.

[24] F. Petropoulos and e. al., "Forecasting: theory and practice," *International Journal of Forecasting,* vol. 38, no. 3, p. 705–871, 2022.

[25] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting,* vol. 36, no. 1, p. 54–74, 2020.

7 APPENDICES

Appendix A: Gantt chart



Appendix B: Subscription plans (manufacturers)